

Reinforcement Learning — From Simulation to Deployment on the Robot Platform Spot

Markus Lejon & Fredrik Sundt



LUND
UNIVERSITY

Department of Automatic Control

MSc Thesis TFRT-9999
ISSN 0280-5316

Department of Automatic Control
Lund University
Box 118
SE-221 00 LUND
Sweden

© 2026 by Markus Lejon & Fredrik Sundt. All rights reserved.
Printed in Sweden by Media-Tryck.
Lund 2026

Abstract

Recent advancements in GPU-accelerated simulation have made it possible to train reinforcement learning policies for robotic control using large-scale parallel data collection. This thesis explores the pipeline from task description to deployment of control policies on the robot platform Spot. The focus is on two important skills needed for future loco-manipulation tasks: robust standing and command-following locomotion. The thesis presents the design choices made when creating an Isaac Lab environment, including the observation space, action space, reward functions, and deployment setup. The resulting policies were tested both in simulation and on real Spot hardware using the Spot RL Researcher Kit. The experiments showed that the trained policies could be deployed on hardware and that successful locomotion was possible. At the same time, a sim-to-real gap was observed, especially in the joint tracking and load-bearing behaviour of the robot. The thesis therefore shows both the potential of the pipeline and the practical challenges involved in transferring reinforcement learning policies from simulation to real hardware deployment.

Acknowledgements

We would like to thank the Department of Automatic Control, the Department of Computer Science, and the RobotLab at LTH for their support and for providing the necessary resources and equipment that made this thesis possible. To our supervisors, Björn Olofsson and Volker Krüger, thank you for your invaluable feedback, insight, and support during the thesis. To Marcus Klang and Hashim Imail, thank you for assisting us in setting up the computing station Bokertov and installing the necessary software inside it. We would also like to thank Oskar Stenberg and Anders Nilsson for suspending Spot in the air during deployment testing.

Contents

List of Figures	9
List of Tables	9
1. Introduction	11
1.1 Research questions	12
1.2 Related work	12
1.3 Individual contributions	13
2. Theory and Background	14
2.1 Reinforcement learning	14
2.2 Reinforcement learning in robotics	17
2.3 The robot platform Spot	19
2.4 Reinforcement learning on the mobile robot Spot	22
3. Environment and Task Design	24
3.1 Configuration of the Isaac Lab environment	24
3.2 Task 1: Robust standing	27
3.3 Task 2: Command-following locomotion	29
3.4 Summary	32
4. Experiments and Results	33
4.1 Experimental setup	33
4.2 Task 1: Robust standing	35
4.3 Task 2: Locomotion	39
4.4 Simulated locomotion test suites	41
4.5 Locomotion deployment	42
4.6 Summary	46
5. Discussion	48
5.1 Development and deployment pipeline	48
5.2 Safety during deployment	49
5.3 Sim-to-real transfer	50
5.4 Future work	51
6. Conclusion	53

Bibliography	54
A. Appendix	59
A.1 Simulated test-suite task descriptions	59
A.2 Training parameters	61

List of Figures

2.1	Spot standing in stowed state, mounted with arm and Jetson computer.	20
3.1	Flowchart of the development chain used in this thesis.	24
4.1	Spot attached to a harness and suspended in the air.	34
4.2	Termination causes during training of the standing policy.	36
4.3	Reward for returning to the default joint configuration during training of the standing policy.	37
4.4	Front left leg behaviour during disturbed deployment. Solid line — measured joint position of Spot. Dashed line — desired joint position from policy.	38
4.5	Front left leg behaviour during simulated standing. Solid line — measured joint position of Spot. Dashed line — desired joint position from policy.	39
4.6	Gait reward during training the two-phase locomotion policy.	40
4.7	Termination causes during training of the two-phase locomotion policy.	40
4.8	Front left leg behaviour during locomotion deployment for continuous forward & sideways walking. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.	45
4.9	Front left leg behaviour during locomotion deployment for various commanded velocities. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.	46
4.10	Front left leg behaviour from pushing during locomotion deployment. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.	47

List of Tables

2.1	Spot leg default joint position and range of motion.	20
2.2	Spot arm joint specifications.	21
2.3	Spot arm default positions.	21
3.1	Observation vector components in the default environment.	26
3.2	Summary of the task designs developed during the thesis.	32
4.1	Normal-suite locomotion policy comparison for 15872 simulated tasks.	41
4.2	Stress-suite locomotion policy comparison of 64512 simulated tasks. .	41
A.1	Numerical configuration parameters for the robust standing policy. . .	61
A.2	Phase 2 curriculum changes in <code>feet_spot_locomotion_env_cfg.py</code> . Phase 2 begins after 60,000 environment steps (2500×24).	62
A.3	Numerical configuration changes between the NVIDIA base policy and the warm-started locomotion model.	63

1

Introduction

As the field of robotics expands from primarily being focused on structured industrial tasks, to a more autonomous paradigm, operating in human environments, robots will be required to perform complex loco-manipulation problems such as manipulating and traversing doors. This requires coordination of movement and manipulation under uncertain contact. Traditional control approaches prove effective in constrained settings with well-modelled interactions, but often require prior knowledge of the door dynamics and struggle to handle disturbances [Zhang et al., 2024; Axelrod and Huang, 2015]. Training a reinforcement learning policy for these complex situations is hard. The pipeline for simulation training to hardware deployment must be established first.

Recent advancements in large-scale GPU-accelerated physics simulations through tools like NVIDIA Isaac Sim allow for expensive parallel data collection [NVIDIA, 2026k; Mittal et al., 2025]. This enables reinforcement learning methods to learn contact-rich manipulation skills, such as in Zhang et al. (2024) where variability of the door’s mechanics and swing direction is handled during operation.

Although recent simulation tools and RL methods have increased the accessibility of robot learning, closing the pipeline from idea to behaviour on hardware remains difficult. Challenges emerge not only in training the policy in simulation but also in establishing a reliable development loop from task formulation, environment design, training, validation, and deployment on real hardware.

This thesis investigates a practical pipeline for training, validating, and deploying reinforcement learning policies on Boston Dynamics Spot [Boston Dynamics, 2026f; Boston Dynamics, 2026b] using Isaac Lab and the Spot RL Researcher Kit [Boston Dynamics, 2026i] together with an NVIDIA Jetson AGX Orin AI computer [NVIDIA, 2026j]. Robust standing and command-following locomotion are developed as prerequisite skills for future loco-manipulation involving both arm and body motion like door opening, which was the initial inspiration and aspiration of this thesis. Furthermore the sim-to-real behaviour is evaluated on hardware.

1.1 Research questions

The following research questions are examined during this thesis:

- How can the process of developing and RL policy trained in Isaac Lab for deployment on real Spot hardware be designed?
- What is required for safe development and deployment of RL policies for Spot?
- How does the trained policy transfer from simulation to reality, reducing the sim-to-real gap?
- What is required for iteratively improving the policy?

1.2 Related work

Reinforcement learning (RL) has grown to become an important and popular tool for robotics control thanks to its ability to handle sequential decision making under uncertainty [Zhang and Mo, 2021; Zheng et al., 2024]. These characteristics make RL well-suited for robot manipulation, and it has been applied successfully by several authors [Tan et al., 2018; Zhang et al., 2024; Tobin et al., 2017; Chen et al., 2022]. Recent work has demonstrated that RL is capable of teaching robust locomotion policies for quadrupedal robots, such as Tan et al. (2018) where a locomotion policy, that was trained in simulation and utilized domain randomization to increase robustness, was able to successfully transfer to a physical version of the robot.

Omotuyi et al. (2024) highlight the importance of a high-quality simulator that minimizes the disparity between simulation and reality. The presence of such tools that close the sim-to-real gap allows for experimentation and variation in approaches that enable creative solutions to problems. Miller et al. (2025) train and deploy a reinforcement learning policy on Spot resulting in Spot running faster than it could with its original controller, while showing the importance of a good model in simulation and how it helps minimizing the sim-to-real gap. Chen et al. (2022) explore the benefits of joint torque control over more conventional joint position control showing increased robustness when disturbed or walking over uneven terrain. RL on quadrupeds in general has proven useful to increase mobility in environments where unexpected behaviours can occur, such as when navigating uneven terrain with rubble, which is difficult to model and control using classical control methods [Boston Dynamics, 2026l]. Huang et al. (2024) use the arm of a quadruped as a stabilizing tail, counteracting some of the imbalances the heavier centre of mass brings. This has, however, limited use during loco-manipulation tasks and keeping the arm in a non-stowed position when moving increases the risk of damage to the relatively sensitive arm. Furthermore, Kumar et al. (2023) equip Spot with prismatic

joints that extend the legs in order to increase stability, however, this could be physically impractical and does not deal with manipulation tasks. Cheng et al. (2023) learn real-world skills such as climbing onto a wall to reach a button or kick a ball. Finally, Zhu et al. (2025) train a reinforcement learning policy for locomotion dynamically using three legs on the quadruped Spot. The remaining leg is used for manipulation tasks together with its arm utilizing classic control methods. Furthermore, the policy is trained in Isaac Lab with a detailed model of Spot where the arm is attached.

In terms of door manipulation specifically, both more classical approaches have successfully been applied in various fashions [Stüde et al., 2019; Gustafsson and Rosqvist, 2023; Tuomisto et al., 2022], as well as reinforcement learning approaches such as Zhang et al. (2024) which utilizes a quadruped with a hook manipulator that manages to, with no previous knowledge of the door, open and traverse both swing directions in a single policy. This work highlights an important step in the pursuit of a general policy which can encounter any door without previous knowledge of its dynamics. However, the detection of the handle was not managed in a general manner, limiting the viability for real-world scenarios.

As can be seen in the literature overview, previous work has demonstrated strong results deploying RL for many robots, including quadrupeds, on various locomotion and manipulation tasks. This thesis focuses on the training and deployment process of RL policies using Isaac Lab and deploying on the Spot platform with its arm attached.

1.3 Individual contributions

The effort has been split evenly throughout between Fredrik Sundt and Markus Lejon and both have contributed equally.

2

Theory and Background

This chapter provides the relevant theory and background regarding reinforcement learning, training of policies for robots and for Spot specifically. The chapter also introduces the mobile robot platform Spot.

2.1 Reinforcement learning

Reinforcement learning (RL) is a subfield of machine learning concerned with learning how to act through interaction with an environment. In contrast to supervised and unsupervised learning, where learning is based on a fixed dataset of labelled or unlabelled examples, RL data are generated through the agent’s own actions and the resulting feedback from the environment. Sutton and Barto (2018) describe RL as learning “what to do—how to map situations to actions—so as to maximize a numerical reward signal”. The acting entity, commonly referred to as the *agent*, learns by trial and error which actions lead to high reward. A central challenge is that actions may have delayed consequences, meaning that an action can be beneficial or harmful even if its effect is not immediately reflected in the reward. Sutton and Barto (2018) therefore emphasize trial-and-error learning and delayed reward as distinguishing features of RL. These properties make RL particularly suitable for sequential decision-making problems with long time horizons. It has been successfully applied to competitive games such as Dota 2, where OpenAI Five defeated the world champion team OG [OpenAI, 2019; OpenAI et al., 2019], and board games such as chess and shogi, where AlphaZero achieved superhuman performance through self-play [Silver et al., 2018]. RL has also grown to become an important and popular approach for robotics control [Zhang and Mo, 2021; Zheng et al., 2024]. In robotics, RL has been used to learn agile locomotion policies that transfer from simulation to real quadrupeds [Tan et al., 2018] to learn interaction skills for door opening and traversal [Zhang et al., 2024].

Since reinforcement learning often handles problems which are sequential, they can be modelled as a Markov decision process (MDP) [Sutton and Barto, 2018]. An MDP is a framework that lets the agent observe the current state, take an action,

observe a reward as the environment reacts and transitions into a new state where the process can be repeated, as can be seen in Algorithm 1 [Sutton and Barto, 2018]. The Markovian assumption means that the current state encapsulates the history, which makes it possible to learn policies that maximize long-term reward [NVIDIA, 2026m].

Algorithm 1 MDP Interaction under Policy π

```

1: Initialize state  $s$ 
2: while not terminal do
3:   Observe state  $s$ 
4:   Sample action  $a \sim \pi(\cdot \mid s)$ 
5:   Execute  $a$ , observe reward  $r$  and next state  $s'$ 
6:    $s \leftarrow s'$ 
7: end while

```

One common algorithm for applying reinforcement learning in robotics is proximal policy optimization (PPO) [Zheng et al., 2024; NVIDIA, 2026k]. PPO is an on-policy policy-gradient method designed to improve training stability by limiting the size of each policy update [Sutton and Barto, 2018; Schulman et al., 2017]. Instead of directly clipping gradients, PPO clips the probability ratio between the new and old policies in the training objective. This discourages updates that move the policy too far from the behaviour that generated the training data, thereby reducing the risk of unstable learning [Achiam, 2018; Schulman et al., 2017].

PPO is often implemented using an actor-critic architecture. The actor maps observations to actions and is therefore the part of the model that directly controls the agent. The critic estimates the value of the current state or observation, which is used during training to evaluate whether an action led to a better or worse outcome than expected. During deployment, only the actor is needed for inference on the robot [Sutton and Barto, 2018; Achiam, 2018].

The issue of robustness is often reduced with *domain randomization*, a technique where different variables such as mass, friction coefficients, positions of elements, etc. are randomized in each episode of training [Tobin et al., 2017]. Robustness refers to how well a trained policy preserves its performance and safety when it is used in environments that are different from those it was trained in.

Key terminology

The following section introduces the key terminology required to understand the reinforcement learning and simulation sections presented later in this thesis. All concepts align with standard RL terminology except *Events* and *Commands*, which are specific abstractions for the simulation environment used in this thesis.

Actions and Policy An action represents the control input an agent can make in its environment. The set of all possible actions define the action space. These vary

depending on the context of the problem. For instance, it could be every legal move on a chessboard, an autonomous car’s steering decision (turning left, turning right or driving forward). In robotic control contexts the action space is often continuous control signals of the joint torques or positions. The *action* a is selected by the *policy* $\pi(a|s)$ given the current *state* s of the world [Sutton and Barto, 2018; NVIDIA, 2026k].

States and Observations A state refers to the underlying physical or simulated condition of the system. This can include the robot pose, joint positions, joint velocities, contact states, and environment parameters. In simulation, these quantities can often be accessed directly from the simulator as exact or near-exact information. On real hardware, however, the true state of the system is not directly available and must instead be estimated from sensor measurements.

Observations are the quantities that are provided to the policy. These may consist of measured or estimated values such as joint positions, joint velocities, base velocity, projected gravity, or processed sensor outputs. In Isaac Lab, the raw simulator state can be processed, randomized, and corrupted with noise before being passed to the policy as observations. In some simulation-based reinforcement learning setups, the observations may also include privileged information that would not be directly available on the real robot, such as object mass, friction coefficients, or door dynamics. Such privileged observations can be useful during training, but must either be removed or inferred before deployment on physical hardware [Zhang et al., 2024; NVIDIA, 2026d].

Rewards The reward function specifies the objective of the reinforcement learning problem by mapping the agent’s behaviour to scalar feedback. The policy is then optimized to maximize cumulative reward. By weighting different reward terms, the designer can shape the behaviour of the agent towards the desired solution. For example, in a standing task, positive reward terms may encourage balance, stable body height, and foot contact, while negative terms penalize excessive torque output, slow recovery, or unsafe behaviour [Sutton and Barto, 2018].

Episodes and Terminations Each iteration of training is denoted as an episode. Terminations decide when an episode should end and the agent should be removed from the environment to adapt the policy based on the feedback it received from the actions that were taken in that episode. Termination can happen for a multitude of reasons, typically catastrophic failures like falling over are terminated as it puts the agent in an unrecoverable state that is heavily undesirable. Other termination clauses include timeouts where the agent is given a certain time interval to complete the task in a timely fashion. Termination can also happen if an agent has successfully completed the given task and there is no more learning required. It is common to separate successful terminations, that can be given positive rewards, and unsuccessful terminations that are punished with negative rewards [NVIDIA, 2026n].

Events Events are often not treated as their own concept in reinforcement learning but are used as a framework in the robotic simulation software. Events are modifications done at an episodic level, to change variables within the system to improve robustness through domain randomization. This could be changing the mass of the robot or changing the friction coefficients of the floor, for example making it slippery. It could also be applying various forces or torques to the robot, requiring it to find ways to adapt and generalize better [NVIDIA, 2026d].

Commands In the simulation environment used in this thesis, Commands are task-related reference signals that specify what the agent is expected to achieve. This could be task goals or specific reference poses it should aim to achieve. Commands define the goal, whilst Actions are the control outputs used to achieve the specified goal and Events modify the environment or robot state [NVIDIA, 2026b].

Curriculum Curriculum learning is a technique in reinforcement learning that aims to adapt the difficulty of the problem which the agent is solving to match the performance; this aims to improve convergence to a better result. A curriculum can also be used to modify reward weights to shape the policy's behaviour and emphasize the importance of different aspects captured in the rewards [NVIDIA, 2026c].

The act of training reinforcement learning policies is split up into three steps. The first is an *initialization step* where the policy network starts out with random weights assigned to its nodes. The specific network architecture can vary between tasks, from simple multilayer perceptron, to transformers and others, depending on the need of the particular task. The second phase, *Rollout*, is where the agent is interacting with the world, using the current policy. At each step, the observations, actions, rewards, and termination signals are stored. Finally, the *policy update* step, is where, using the data gathered during rollout, the neural network updates to maximize the expected reward. When running this loop over time the policy learns to optimize the positive rewards and minimize the negative ones [NVIDIA, 2026k].

2.2 Reinforcement learning in robotics

In order to train a reinforcement learning policy for control of robots, a trial-and-error process for data collection is required. Real-world interaction data can be expensive to collect, and in many cases it would be unsafe to do so [NVIDIA, 2026l]. To tackle these challenges robots are first trained in simulation and then transferred to real hardware. One solution to this problem is presented by NVIDIA with their simulation environment Isaac Sim and their reinforcement learning framework Isaac Lab.

Isaac Sim & Isaac Lab

Isaac Sim is a physics simulator developed by NVIDIA, built for development and testing of robotic solutions in a physically accurate virtual environment [NVIDIA, 2026i; NVIDIA, 2026q]. Built on top of Isaac Sim is NVIDIA’s robot learning framework Isaac Lab. It extends the simulator with capabilities for training control policies using learning-based methods such as reinforcement learning and imitation learning. In this setup, Isaac Sim advances the environment by one time-step at a time, producing observations that are passed to the policy. The policy then generates actions, which are applied to the simulator in the next step, forming a closed interaction loop [NVIDIA, 2026d]. The GPU-focused design enables a large number of environments to be simulated in parallel, allowing for efficient large-scale data collection across many instances of the same task. This is essential for achieving sample-efficient reinforcement learning [NVIDIA, 2026h; NVIDIA, 2026f; NVIDIA, 2026p; NVIDIA, 2026e; NVIDIA, 2026g; Mittal et al., 2025].

Sim-to-real gap

A simulation tool like Isaac Sim, no matter how accurate, will always present a certain gap between simulation and reality. A few notable challenges include model mismatch and physical parameters like mass, ground friction or joint parameters like stiffness or load that do not match reality. Furthermore, there is an observation mismatch where the simulation has perfect signal information, whereas the actual sensors on the hardware introduce noise. Moreover, a policy output by itself does not produce a value that necessarily have to have a physically possible position in the joint space it controls. This presents a potential *sim-to-real gap*, where the differences from an imperfect model of reality make actions not translate correctly.

Training reinforcement learning policies in Isaac Lab

To train a reinforcement learning policy using Isaac Lab, a defined *environment* is required for the training to take place in. This environment includes the *scene*, which specifies all objects and entities that should exist in the simulation. This includes objects such as the robot which the training should take place for, the ground-plane or terrain for the robot to move on, a lighting source, obstacles and interactive elements such as a door for the robot to engage with [NVIDIA, 2026o].

One central part of the *scene* is the specification of the robot embodiment. These are defined as a collection of rigid body elements connected by joints, forming a kinematic structure. There exists various predefined models inside Isaac Lab that can be initialized and deployed directly in the simulation environment. But it is also possible to configure custom robots for a specific scenario. This includes specifying the model to be spawned, the initial position of the robot and its joints, as well as defining what parts of the robot should move and how they accomplish this movement [NVIDIA, 2026o; NVIDIA, 2026r].

In addition to the scene, the environment defines the different parameters presented in the subsection *Key terminology*, as well as various configuration settings that control both the simulation and the training process. This includes the observation space, which determines the information available to the policy at each time-step, such as information about the robot itself and the environment it interacts with. The action space specifies how the policy can interact with the robot, for example by controlling joint positions or torque outputs. Reward functions quantify the robot's behavior and guide learning towards desirable outcomes. The environment further defines termination conditions for the current episode, as well as event-based changes applied at different time points or in response to specific situations. The termination clauses primarily cover successful task completion or catastrophic situations, such as the robot falling over. Events handle environment randomization, ranging from physical parameters to initial state conditions. These variations improve the robustness of the resulting policy and often help close the sim-to-real gap [NVIDIA, 2026a; NVIDIA, 2026b].

2.3 The robot platform Spot

The Spot robot platform is an agile quadruped mobile robot, see Figure 2.1 for a visual representation, which is developed by Boston Dynamics [Boston Dynamics, 2026a; Boston Dynamics, 2026b; Boston Dynamics, 2026f]. It can be attached with additional payloads. In this thesis, Spot is equipped with the Spot Arm [Boston Dynamics, 2026g] and the NVIDIA Jetson AGX Orin [NVIDIA, 2026j] computer module mounted with the Boston Dynamics Spot RL mounting kit [Boston Dynamics, 2026i].

Hardware specification

Base platform. Spot has a length of 1100 mm, width of 500 mm, and a default walking height of 610 mm. It has a weight of 32.5 kg without any additional payloads, and a maximum speed of 1.6 m/s. Spot has four legs that each have 3 degrees of freedom (DOF), which together result in a total of 12 DOF. Each hip consists of two actuators, and each knee consists of one actuator. The hip joints are defined as HX and HY, corresponding to the two planes of rotation. In the robot's base frame, the x-axis points forward, the y-axis points to the left, and the z-axis points upwards [Boston Dynamics, 2026a; Boston Dynamics, 2026b; Boston Dynamics, 2026k].

Boston Dynamics states in Boston Dynamics (2026a) that the movement of each joint is defined as follows:

- Hip Joint X-axis (HX): +/- 45 degrees from vertical (45 degrees of internal and external rotation from vertical).
- Hip Joint Y-axis (HY): +/- 91 degrees with 50 degree bias from vertical (flexion/extension).



Figure 2.1 Spot standing in stowed state, mounted with arm and Jetson computer.

- Knee (KN) : +/- 14-160 degrees from straight (flexion/extension range from 14 to 160 degrees).

The range of motion of the leg joints, as well as their default position used during training and deployment can be seen in Table 2.1.

Table 2.1 Spot leg default joint position and range of motion.

Joint Axis	Default Position	Min Position	Max Position
Front Left HX	5.73°	-45°	45°
Front Right HX	-5.73°	-45°	45°
Hind Left HX	5.73°	-45°	45°
Hind Right HX	-5.73°	-45°	45°
Front Left HY	51.57°	-51°	131°
Front Right HY	51.57°	-51°	131°
Hind Left HY	63.03°	-51°	131°
Hind Right HY	63.03°	-51°	131°
Front Left KN	-85.94°	-160°	-14°
Front Right KN	-85.94°	-160°	-14°
Hind Left KN	-85.94°	-160°	-14°
Hind Right KN	-85.94°	-160°	-14°

Spot arm. The Spot arm is attached as a payload on Spot, having 6 DOF plus an additional gripper. The arm weighs 8 kg, is 984 mm long and can reach a height up to 1800 mm when attached on the robot. It has a lifting capacity of up to 11 kg, and a pulling capacity of 25 kg on carpet [Boston Dynamics, 2026c].

Table 2.2 is based on [Boston Dynamics, 2026h] that describes each arm joint, their range of motion, and maximum torque. Furthermore, each joint has the same maximum velocity of 10 rad/s.

Table 2.2 Spot arm joint specifications.

Joint Axis	Description	Range	Max. Torque
SH0	Shoulder rotation	$-150^\circ/180^\circ$	90 Nm
SH1	Shoulder pivot	$-180^\circ/30^\circ$	90 Nm
EL0	Elbow pivot	$0^\circ/180^\circ$	90 Nm
EL1	Elbow rotation	$-160^\circ/160^\circ$	23 Nm
WR0	Wrist pivot	$-105^\circ/105^\circ$	23 Nm
WR1	Wrist rotation	$-165^\circ/165^\circ$	23 Nm
FIX	Gripper pivot	$-90^\circ/0^\circ$	11 Nm

When the arm is not actively in use during operations, it sits in its resting or *Stowed* position. The joint configuration of this position can be seen in Table 2.3 and is gathered through sensor readings from the robot.

Table 2.3 Spot arm default positions.

Joint Axis	Description	Stowed Position
SH0	Shoulder rotation	0°
SH1	Shoulder pivot	-180°
EL0	Elbow pivot	180°
EL1	Elbow rotation	90°
WR0	Wrist pivot	0°
WR1	Wrist rotation	-90°
FIX	Gripper pivot	0°

Spot SDK & API

Boston Dynamics provide a software development kit (SDK) for Spot. This is used to enable development of applications and payloads for the platform. It provides a multitude of features such as documentation that explains concepts needed to understand the system, from robot hardware to various implementation details. Furthermore, the SDK provides a Python client library that contains several examples for controlling and receiving sensor data, both from the base platform and attached payloads. An API protocol is also defined to communicate with Spot, allowing integration with other programming languages than Python [Boston Dynamics, 2026j].

Robot State Stream. The Spot SDK exposes the state of the robot through the robot state stream service as part of the Joint Control API. This interface provides essential information about the robot and is suitable for real-time communication. This includes information about the state of the robot, measurement of the joint position, velocity, torque, as well as the pose and linear- and angular velocity of Spot. It is also possible to request the full robot state which provides additional information about Spot, however, at a lower rate of communication. The information gathered from the robot state stream is used to build the observation vector which is then passed to the policy’s inference session [Boston Dynamics, 2026d; Boston Dynamics, 2026e].

Robot Command Stream. The Spot SDK exposes the robot command streaming service as part of the Joint Control API. This interface allows the user to send movement commands to each of the robot’s joints via an external application. Through the stream it is possible to send the desired joint position q_d , angular velocity $\dot{q}_d$, and torque τ_d , as well as the proportional position error gain k_p and proportional velocity error gain k_d . Furthermore, it is possible to include additional information regarding the time frame of the desired command as well as safety limits regarding the joint velocity [Boston Dynamics, 2026d; Boston Dynamics, 2026e].

When sending a joint-level robot command through the robot command stream it does not directly control the servo motors of the joints. Instead, the total desired torque output τ of the joint motors is calculated by the underlying PD controller from the desired position, velocity, and torque together with the measured joint position q and velocity $\dot{q}$. The control law for the PD controller is as follows [Boston Dynamics, 2026d]:

$$\tau = k_p \cdot (q_d - q) + k_d \cdot (\dot{q}_d - \dot{q}) + \tau_d. \quad (2.1)$$

Example usage of the API. In this thesis, the output of the reinforcement learning policy a_π represents the joint position offsets relative to the robot’s default pose. From this value, the desired joint positions, q_d , are calculated by post-processing the output. This is done by applying an action scale a_{scale} to the policy action and adding it to the default joint configuration $q_{default}$, shown in the the following equation:

$$q_d = q_{default} + a_{scale} \cdot a_\pi \quad (2.2)$$

2.4 Reinforcement learning on the mobile robot Spot

Spot RL Researcher Kit & NVIDIA Jetson AGX Orin

In order to facilitate reinforcement learning research on the Spot platform, Boston Dynamics, in collaboration with NVIDIA, released the Spot RL Researcher Kit [Omotuyi et al., 2024]. The kit provides the necessary hardware and software in-

infrastructure needed for the development, training and deployment of reinforcement learning policies on Spot [Boston Dynamics, 2026i].

A key component of the RL Kit is the joint-level API [Boston Dynamics, 2026d] license which grants access to lower-level control of the robot’s motors and actuators. This is relevant for reinforcement learning since the policy can control each actuator of the robot individually, instead of relying on the existing control loops exposed through the Boston Dynamics SDK.

Combining this access with a detailed model in Isaac Sim, together with the training capabilities of Isaac Lab, enables the training and simulation of an RL policy and its subsequent deployment on the real hardware to control the robot.

Furthermore, the kit is shipped with parts for hardware mounting and an interface connection, which allows the mounting of an NVIDIA Jetson AGX Orin [NVIDIA, 2026j] directly on Spot. The Jetson computer module, developed for AI computations, adds the necessary computational capability to power the inference computations in real time.

The kit also includes deployment code that acts as a bridge between the trained RL policy and Spot’s command API, intended to run on the mounted Jetson computer. This software provides the necessary infrastructure for communicating with Spot and for executing the policy inference session. Within the control loop, sensor data are obtained via the *Robot State Stream* which are used to build the desired observation vector. This vector is then passed to the policy’s inference session, which returns an action vector. The actions subsequently undergo the required post-processing. Finally, the processed output is transmitted through the *Robot Command Stream*, after which Spot’s internal PD controller converts them into actuator torque that drives the robot’s joints.

Overall, the Spot RL Researcher kit provides the necessary components to deploy an RL policy on Spot. The simulation environment Isaac Sim, together with the training framework of Isaac Lab, enables training of reinforcement learning policies in a simulated environment. Combined with the Spot platform, these components establish a complete pipeline for training, evaluating, and deploying reinforcement learning policies [Boston Dynamics, 2026l; Ackerman, 2024].

3

Environment and Task Design

The implementation was developed iteratively around two prerequisite skills for future door-opening loco-manipulation: robust standing and command-following locomotion. Robust standing was used as a low-risk first deployment target and as a recovery behaviour. Locomotion was then developed through three approaches: direct training, blending with the standing policy, and warm-starting from a pre-trained locomotion policy. The following sections describe the environment configuration and the task-specific design choices.

An illustration of the development process to design, train, and deploy reinforcement learning policies on Spot used in this thesis is highlighted in Figure 3.1. The initial task design informs how the environment in Isaac Lab is set up in order to train a policy. The policy is thereafter evaluated in simulation and iteratively improved upon. When a sufficient performance is achieved in simulation, hardware deployment can be conducted, whereafter the resulting behaviour is analysed.

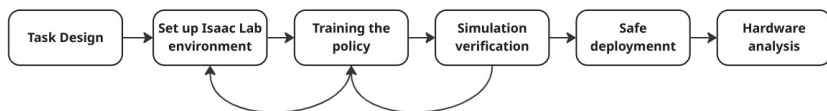


Figure 3.1 Flowchart of the development chain used in this thesis.

3.1 Configuration of the Isaac Lab environment

As discussed in Section 2.2, the environment is a key building block of how Isaac Lab is structured. This section will present the choices that were made when shaping the particular experiments that are performed in this thesis.

The scene

The scene contains all elements needed to create an instance of the complete agent training environment. This will vary depending on what the agent is trying to learn. The default scene contains the following elements.

- A plane, flat ground with reasonable default values for physics parameters such as friction.
- A robot model, in this thesis a model of Spot with its arm will be used.
- A light source in order to see the environment when rendered.
- Contact sensors placed on each joint of the model in order to detect collision forces.

Actions

As mentioned in subsection *Hardware specification*, the robot contains 19 joints in total. The legs have a total of 12 joints split up equally across its four legs, the arm has 6 joints plus 1 for the gripper. The default action paradigm chosen is the joint positions of both the leg and the arm joints. This was chosen since it follows the convention set in Isaac Lab and NVIDIA’s locomotion example [Omotuyi et al., 2024]. This is further motivated by Chen et al. (2022) that considers joint action to be one of the two main locomotion modalities. Other implementation options that could be explored include using relative joint position, where commands are processed relative to the current position instead of the absolute value. Another option is joint velocity control, which issues velocity commands instead of position commands. Finally, joint effort control can be utilized that directly send force commands to the motors, which was shown to be successful by Chen et al. (2022). As described in subsection *Spot SDK & API* the actions output by the policy network are scaled with an action scale parameter a_{scale} that influences the granularity of the output. In this thesis this factor was set to 0.2 since it was used in other examples and was therefore seen as a sensible default value for all joint position actions.

Commands

The command specifications will be heavily influenced by the specific task the policy is being trained to perform. Hence no default command is specified in the base environment.

Observations

The observations vector implemented in the default environment contains the elements presented in Table 3.1. Default observations include the linear and angular velocity of the base, projected gravity, joint positions and velocities, and the last performed action which helps stabilize learning. Each of these observations comes with

Table 3.1 Observation vector components in the default environment.

Observation component	Dim	Purpose
Base linear velocity	3	Describes translational body motion.
Base angular velocity	3	Describes rotational body motion.
Projected gravity	3	Gives body orientation relative to gravity.
Joint positions	19	Gives current joint configuration.
Joint velocities	19	Gives joint motion state.
Previous action	19	Helps stabilize policy behaviour over time.
Total	66	Full policy observation vector.

an added noise term to simulate the noise of real sensor data. This addition helps improve stability when deploying on real hardware. Each task can be equipped with its own additional observations that may include privileged information unavailable to the robot in reality. One example is the work of Zhang et al. (2024), where their teacher model has access to door dynamics during training. A smaller student model is then trained without this information and instead learns to infer the swing direction of the door upon interaction. For tasks containing commands its often sensible to include these in the observation vector.

Events

The reset event is implemented by default to trigger domain randomization upon the agent restarting. The only implemented event in the base environment is an event to reset the agent's joints to their default position. The specific domain randomizations will differ for each task.

Rewards

The primary lever to influence the policy towards desired behaviour are the rewards. The only two rewards given by default are a positive reward for staying alive and a large negative reward for terminating. These can be seen as universal and always desired. Other rewards are task specific and are therefore implemented accordingly.

Termination

There exist two termination conditions by default, one for timing out and one for collision of the body, upper leg, or arm with the ground. Termination via time out is triggered when the agent has existed for the entire duration of its training instance. This is used to give a finite end to a continuous task that has not been terminated by other factors. It ensures that the agent does not spend excessive time on a single episode and instead restarts its learning process with a different configuration. The collision clause is used to terminate any unwanted impacts between the ground and the robot. This is to mitigate damage to the hardware and to avoid unrecoverable

situations, for example as a result of losing balance and falling over. These situations have to be treated as critical failures and a state from where the agent should no longer learn.

Curriculum

No curriculum is set up in the default environment. A potential candidate could be to randomize terrain to ensure the agent performs the specified task on non-flat terrain as this presents a more challenging task that increases robustness.

Model of Spot with arm

Isaac Lab does not natively implement a model of Spot with the arm payload attached, only a model of Spot without the arm. In Isaac Sim there exists one Spot model without the arm, and one Spot model with the arm. These models can be imported and configured to be used for training inside Isaac Lab. In this thesis, however, a model of Spot with the arm payload attached provided by the RAI Institute was used during the development process [Miller et al., 2025]. This version has been used in previous training in Isaac Lab with deployment on real hardware, validating that this model of Spot is close enough to reality to not present a large sim-to-real gap. Furthermore, the model contained key features that made it suitable for the environment it was going to be trained in. The model was configured to use the default configuration presented in Section 2.3.

3.2 Task 1: Robust standing

In order to test the complete pipeline while reducing the risk of damage during deployment, the first policy developed was a robust standing policy. The objective of the policy is to ensure the robot maintains balance whilst handling external disturbances without falling. Only sections that have noteworthy changes from the default environment will be discussed.

The scene

The environment is configured with functionality to generate non-planar terrain. The level of difficulty is regulated by a curriculum-based schedule, when the model's performance improves, the task complexity is increased incrementally to sustain continued learning and performance enhancement.

Commands and expansion of observation vector

To maintain compatibility with the observation vector used for later locomotion policies, the standing task includes a three-dimensional velocity command expressed in the robot base frame. This corresponds to desired velocity in the x- and y-directions and the desired yaw rate in the robot's base frame. However, for the robust standing task this command is always set to zero. The policy is therefore trained

to maintain a stationary posture while preserving the same observation structure as the locomotion policies. This expands the total observation vector to 69 dimensions.

Events

The events that were implemented for the standing task were to assign random static and dynamic friction to each body part of the robot to increase robustness. The values of the static friction are between 0.2 and 1.4, and for the dynamic friction they are between 0.15 and 1.2. The code generates 128 *buckets* of different values that each environment gets assigned at startup. This results in each instance being trained with a slightly different surface interaction. There is a second event that varies the mass of the robot within the range of ± 4 kg from the default value. This represents about 10% of the total body weight and about 25% of the base's weight, a significant change in dynamics that the robot must adapt to.

Reset events Beyond the startup variables discussed previously, multiple disturbances are placed upon the robot on reset. The robot's position in the environment is randomized, including its x- and y-position as well as its yaw rotation. The robot is initialized with linear velocities in the x, y, z direction, as well as angular velocities about the roll, pitch, yaw axes, resulting in the robot learning to recover from disturbances gracefully. An external force and torque is applied to the base of the robot and to each joint. The position of each joint is also offset from its default position. Furthermore, a linear velocity is applied to the robot to simulate an external push. This perturbation is triggered periodically to encourage the learning of recovery behaviour from external disturbances beyond the initial recovery.

Rewards

The reward functions for this task can be broken down into several goals.

- Do not terminate by the body or upper legs making contact with the ground.
- Return to the default established position whilst standing still.
- Achieve the two previous items in a graceful manner that handles recovery from difficult positions.

These goals are reflected in how the reward functions are structured, having the first two rewards being dedicated to not terminating and linearly scaling with staying alive longer. This was found to prevent some tendency for early termination analogous to the agent suicide discussed by Martin et al. (2016) where some training attempts got stuck in a local maximum with early termination. Secondly, a large reward is given proportional to the default joint positions. The larger weight was motivated by the desire for the agent to recover to the default position after stabilizing. Previous attempts found the agent often recovering gracefully but stayed in the recovered position instead of returning to its default position. Rewards are

also given for tracking linear and angular velocity as compared to a given reference velocity, this reference is always set to zero, encouraging the agent to recover to a state where no velocity is present on the base. Lastly, a small reward per foot making contact with the ground is also given.

The penalties are configured to mainly serve the third goal, penalizing the lack of action smoothness, base motion, non-flat orientation, deviance from the standing height, and foot slipping. Joint movements are restricted to discourage the agent from rapid small changes, causing non-smooth behaviour. Finally, the arm joints are all additionally penalized for acceleration, velocity and heavily for deviation from the initial stowed position, discouraging the use of the arm as a balancing mechanism.

3.3 Task 2: Command-following locomotion

Commands

The second control policy that was developed was a locomotion task, a crucial step towards the loco-manipulation goal. The agent was commanded to track a target velocity vector, replicating the approach of Omotuyi et al. (2024), Miller et al. (2025), and Zhu et al. (2025).

At the start of each episode, and periodically throughout rollout, a new target command was sampled from a continuous uniform distributions for the linear velocities in the robot’s local x- and y-direction, as well as for the angular yaw velocity. These target velocities were then provided to the policy as part of its observation space, matching with the robust standing policy.

The objective of this task was twofold. First, the agent had to learn to rapidly accelerate, decelerate, and reorient its body so its centre-of-mass velocity closely matched the commanded target velocity. Secondly, it had to achieve this while maintaining balance and avoiding excessive joint torques or unstable motions. As a result, the learned behaviour became more dynamic and diverse, since the agent was continuously required to adapt to changing velocity commands while simultaneously handling externally applied disturbances.

Two-phase locomotion

Rewards. To accomplish the task specified by the command, the rewards can be divided into three distinct categories.

1. Track the provided command as closely as possible, both for the linear and angular velocities.
2. Achieve the tracking with a graceful gait that remains steady.
3. Keep the arm in a steady and stowed position, see Figure 2.1.

The agent is rewarded based on how closely the base velocity matches the commanded velocity. The linear velocity reward penalizes the Euclidean tracking error in the horizontal plane, while the angular velocity reward penalizes the absolute yaw-rate tracking error. This is a common way to measure these tracking errors and was already implemented in Isaac Lab.

Several different reward functions and weights were tested in order to achieve the second goal. The most important of these is the Gait reward, a function that encourages the robot to develop a diagonal gait pattern where the front-left and hind-right legs move in coordination, and likewise the front-right and hind-left pair. Other reward functions were designed to encourage keeping the foot pair in the air and with sufficient foot clearance whilst keeping the variance of the gait to a minimum. The action smoothness, base orientation, and foot slip penalties from the standing environment are all still present here alongside small penalties for joint acceleration, torques, and velocities to discourage unnecessary movement that do not serve the goal. Finally, like in the robust standing example, the arm is penalized for moving out of its default stowed position.

Two-phase architecture details. After empirical tuning and experimenting with different weights and structures an additional curriculum was introduced, separate from the previously discussed terrain curriculum. The purpose was to separate the learning of basic gait structure from the later refinement of velocity-command tracking. This results in splitting the training into two different and distinct phases.

In the first phase, the primary objective was to establish a stable and repeatable walking pattern. To achieve this, the gait reward term was intentionally assigned a relatively high weight in the overall reward function, increasing the influence during early training. This encouraged the policy to first learn coordinated leg motion before being driven towards more aggressive velocity tracking. Specific weight changes are published and can be found in Appendix A.2.

In the second phase, the gait reward weight was reduced and the command-tracking terms were increased. The aim is to shift the optimization from the learned stepping pattern towards following the commanded linear and angular velocities accurately. This transition therefore allowed the policy to first learn a stable locomotion structure before being optimized for tracking performance.

Blending policies

A problem that occurred during the development of the locomotion policy was a tendency for the model not to stand still when no command was given. The agent gained more reward from slowly gaiting forward, even when commanded to have no base velocity. This was initially counteracted with rebalancing of weights and continued learning, but was found to reduce the overall robustness and worsened walking performance.

This issue was instead solved with a combination of the standing and locomotion policies using a threshold-based policy selector. The selector switches to the

standing policy when both the commanded velocity, base velocity, and yaw rate are deemed small enough. The locomotion policy is used when any of three values are too high. The switching logic is summarized in Algorithm 2.

Algorithm 2 Threshold-based blending of standing and locomotion policies.

Require: Observation o_t , velocity command u_t , previous active policy π_{active}

```

1:  $c \leftarrow \|u_t\|_2$ 
2:  $v \leftarrow \sqrt{(v_x^{\text{base}})^2 + (v_y^{\text{base}})^2}$ 
3:  $\omega \leftarrow |\omega_z^{\text{base}}|$ 
4: if  $c > c_{\text{th}}$  or  $v > v_{\text{high}}$  or  $\omega > \omega_{\text{high}}$  then
5:    $\pi_{\text{active}} \leftarrow \pi_{\text{locomotion}}$ 
6: else if  $c \leq c_{\text{th}}$  and  $v < v_{\text{low}}$  and  $\omega < \omega_{\text{low}}$  then
7:    $\pi_{\text{active}} \leftarrow \pi_{\text{standing}}$ 
8: else
9:   Keep previous  $\pi_{\text{active}}$ 
10: end if
11:  $a_t \leftarrow \pi_{\text{active}}(o_t)$ 
12: return  $a_t$ 

```

The interval between the lower and upper base-velocity thresholds acts as a hysteresis region. This prevents rapid oscillation between the two policies when the robot is close to the switching boundary. In practice, the threshold values had to be tuned for each combination of standing and locomotion policies, since different policies produced different transient behaviours near zero command.

Warm-starting from base policy

NVIDIA has published an example locomotion policy [Omotuyi et al., 2024] for Spot without the arm. In order to avoid requiring the blending of policies, a second locomotion attempt was made where the arm joints were removed from the action space in order to match with the locomotion policy developed by NVIDIA. This locomotion policy did not replicate the stand-still gaiting behaviour. However, it proved less stable than desired when simulated on the model with an arm, often tilting too heavily because of the changes in the dynamics. This was rectified by letting the model gain experience of this new embodiment by warm-starting training from the existing weights. This training increased the stability of the locomotion over time, but needed rebalancing of some weight functions in order to not reintroduce the stand-still gaiting behaviour. Further analysis of the comparative performance of this model will be discussed in Chapter 4. Specific weight changes can be found in the reproducibility table in Appendix A.2 of this thesis for interested readers and for future work.

Self-collision fine-tuning

After the main training phases, an additional fine-tuning stage was introduced to reduce self-collision tendencies in the locomotion policies. During simulation, both locomotion solutions occasionally placed the feet too close together, causing collisions. To address this, self-collision handling focused on the lower limbs was enabled during a short additional training phase.

After fine-tuning, the observed foot placement behaviour improved and the tendency for the feet to move into self-colliding configurations was no longer present. This fine-tuning step was therefore used as a final corrective stage before comparing the locomotion policies in the simulated test suites.

3.4 Summary

Below is a summary of the different tasks and policies developed in this thesis.

Table 3.2 Summary of the task designs developed during the thesis.

Task	Action space			Command	Main objective
Robust standing	19	joint	posi- tions	Zero base veloc- ity	Disturbance recovery and sta- ble standing.
Two-Phase	19	joint	posi- tions	Sampled base velocity	Command-following loco- motion while maintaining balance.
Blended policies	19	joint	posi- tions	Threshold- based switch	Combine dynamic locomotion with stable standing.
NVIDIA warm- start	12 leg	joint	posi- tions	Sampled base velocity	Adapt pretrained locomotion to Spot with arm.

4

Experiments and Results

This chapter presents the results from the policy training, the user commanded simulations in Isaac Lab, and the hardware deployment. The process of generating the reinforcement learning policy for each task was done iteratively, where the structure and reward weights were modified during development to get the desired result. The final policy from each task will be presented and the corresponding reward structure is documented in the appendix. The user commanded simulation is a script to provide a command signal to the policy, together with the rest of the observation vector, which calculates an action that is executed on a single simulated robot. The hardware deployment is executed using a modified version of the deployment code from the Spot RL Researcher kit. The modifications are there to accommodate the extra degrees of freedom that come with the attached arm, as well as a way to freeze specific joints in place to aid during early testing.

The results will include graphical information of the training process and of the policy output and robot position during simulation and deployment. The information in the graph as well as observations of the behaviour will be discussed.

The complete implementation for training, evaluating, and deploying RL policies on Spot, including the Isaac Lab environments, the user commanded simulation, the deployment framework, and evaluation scripts, can be found in the GitHub repository associated with this thesis [Lejon and Sundt, 2026].

4.1 Experimental setup

Hardware deployment on Spot

When first deploying a trained reinforcement learning policy on the real Spot hardware, the robot was suspended in the air, as shown in Figure 4.1. The purpose of the suspension system was to reduce the risk of damage in the case of a catastrophic policy failure or loss of balance during early testing. The harness provided a safety layer, preventing impact with the ground, protecting the hardware and actuators of the robot as well as the surrounding environment.

The suspension setup allowed for a gradual testing process. Initially Spot could be started completely off the ground in order to observe the policy behaviour without any contact with the ground. The robot could then be lowered towards the ground until it eventually supported its full body weight on its own, while the harness remained attached as a safeguard. Additional experiments could be conducted with the robot starting fully from the ground while still attached to the harness in case of any failures such as loss of balance.

However, both simulation and deployment showed that the robot behaves differently when suspended compared to when standing on the ground. The difference is expected since the actuators and body dynamics change significantly when the robot is not affected by the reaction forces of the ground and load of the actuators. Additional effects of the suspension include possible misalignments of the body orientation in roll and pitch as well as swinging motions of the suspension system caused by the motion of the actuators since the robot was not constrained by the friction of the ground. This behaviour would affect the observations which in turn could affect the output of the policy and also how the executed action impacts the robot's state. Consequently, suspended tests could not completely represent the behaviour observed during normal operation. However, it could be seen as a useful tool for validating safety of the policy behaviour before conducting tests on the ground.



Figure 4.1 Spot attached to a harness and suspended in the air.

User commanded simulation

There exists simulation capabilities within Isaac Lab that can be used to test and verify a trained reinforcement learning policy. This simulation loads a trained policy and executes it in the same environment configuration that was used during training. During execution, the environment generates the observation vector, including the robot velocity command, which is passed to the policy. The policy then outputs an action that is applied to the robot.

This simulation environment was useful for testing and verifying the behaviour and performance of the trained policy. However, a limitation of this setup was that the user did not have direct control over the command passed to the policy. To address this, a custom simulation script was developed that also loads the trained policy using the same environment configuration, but where the user could manually specify the command input. This provided improved control of the robot, enabled testing of specific scenarios, and allowed experiments that more closely resembled real-world deployment conditions. Allowing the user to provide commands to the agent also reflects how the system operates during hardware deployment. Additionally, the functionality to apply an external force to the robot was implemented in order to evaluate the robustness of the policy against disturbances such as pushing or external perturbations.

However, the simulation environment still runs entirely within Isaac Lab, using the same physics engine and configurations as during training. As a result, potential sim-to-real discrepancies related to modelling assumptions, contact dynamics, sensor noise, or physics inaccuracies may not become apparent during these tests. Therefore, caution is still required during deployment on Spot.

4.2 Task 1: Robust standing

As described in Section 3.2, the robust standing skill was designed around three main objectives: staying alive, staying still, and recovering to a predefined pose. These objectives were reflected in the reward structure, termination conditions, and disturbance events introduced for the task. The following sections examine illustrative graphs from the training process and discuss when these behaviours emerge. As discussed in Section 2.1, robustness in learning-based legged locomotion is commonly encouraged and evaluated by exposing the policy to randomized physical parameters and external perturbations during training or testing [Tan et al., 2018]. For the robust standing task, a policy is therefore considered robust if it can maintain balance, avoid unsafe termination, and recover towards a stable standing posture after disturbances such as initial state perturbations, externally applied forces, or changes in contact conditions.

Training progression and qualitative results

As can be seen in Figure 4.2, the model initially handles external disturbances poorly and shows no recovery behaviour. Consequently, episodes terminate as a result of contact with the ground, initially from the body making contact with the ground. Around training step 650 the model begins to learn the recovery behaviour and the fraction of agents that reach the timeout condition starts increasing, where the model finds recovery solutions that reduce ground contact with the non-foot part of the leg. The model shows a 3 % non-timeout termination which is deemed an acceptable result. While observing the policy during rollout, the rare case of a contact termination was found to be caused by the combination of the hardest domain randomization factors.

Figure 4.3 shows the reward signal associated with returning to the default joint position. This reward increases steadily during training and eventually plateaus, indicating that the policy learns to avoid termination and to recover towards the desired standing posture after disturbances.

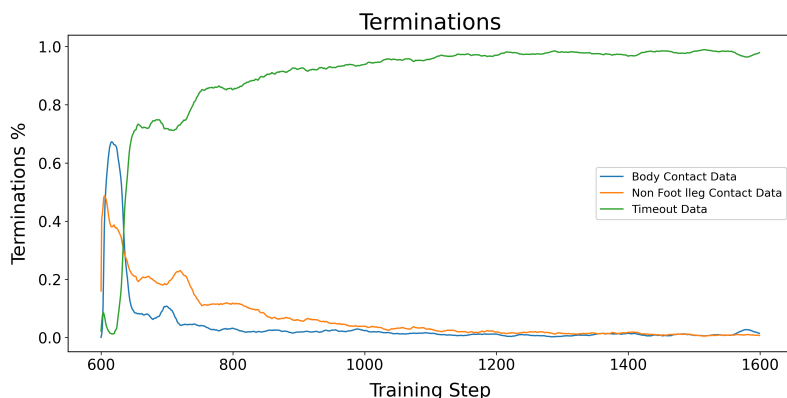


Figure 4.2 Termination causes during training of the standing policy.

Simulation and hardware deployment comparison

The trained standing policy was first evaluated in simulation and subsequently deployed on Spot. The hardware deployment began with the robot suspended in the air using the harness, allowing the policy to be tested without any contact with the ground. In this configuration, the policy maintained a stable standing position which it consistently returned to after external disturbances were applied to the body and joints. After the initial test, Spot was gradually descended towards the ground until the robot fully supported its own weight without assistance from the suspension system. The policy was then tested without the harness, where Spot maintained a stable standing posture and demonstrated robustness against external disturbances.

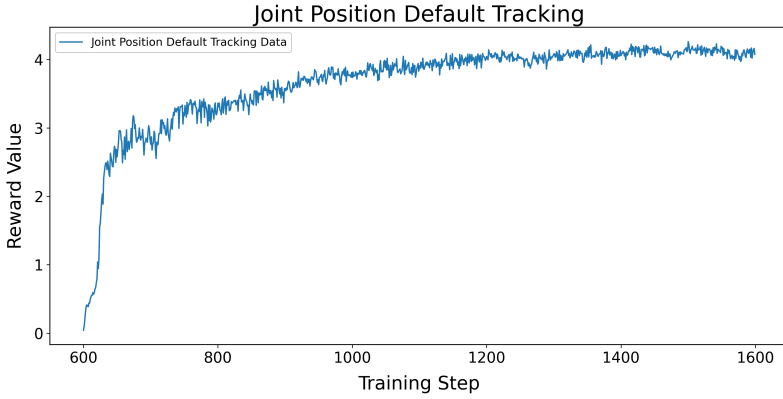


Figure 4.3 Reward for returning to the default joint configuration during training of the standing policy.

Specifically, Spot consistently recovered from pushes on the body and pulling of the joints without falling. Similar behaviour was observed in the simulation, where the robot also recovered from externally applied forces while maintaining a stable position.

Although the policy demonstrated a stable and robust behaviour in both simulation and deployment, the robot assumed a slightly lower standing position than the expected default robot configuration. This behaviour was observed in both simulation and on hardware, although it was more pronounced during deployment on the real robot. This position was not caused by the policy outputting a lower joint target position. Instead, a static error between the commanded joint position and the measured position was observed, particularly for the knee joints. The exact cause for this behaviour remains unresolved. One possible explanation is a sim-to-real discrepancy caused by inaccurate modelling of the knee joint actuators. As mentioned by Miller et al. (2025), the Spot knees use a remotized lead-screw joint [Boston Dynamics, 2026d] that required the most accurate modelling in their sim-to-real comparison and therefore contribute to the sim-to-real-gap.

However, since the same behaviour was also observed in simulation, the issue may not originate exclusively from a mismatch between simulation and reality. Since the simulation environment uses the same model and physics engine as during training, it is possible that the policy has instead learned this behaviour where a static error arise and the standing height is lower than expected. This interpretation is reasonable considering the control structure of the actuators used in both Isaac Lab and of the hardware. The policy outputs desired joint positions, which are converted into joint torque through a PD-controller, see Equation 2.1. Under external loads, such as gravitational force acting on the robot, a static positional error is expected since the controller lacks an integral term and therefore cannot com-

pensate for the resulting steady-state error. This explanation is further supported by the result from the suspension experiment, where the measured joint positions were significantly closer to the commanded positions as a result of the reduced external load on the joints. To add to this, when increasing the proportional gain of the knee joints in simulation, the static error is reduced as a result of the increased torque output for a given desired and measured joint position.

Consequently, it remains unclear to what extent the observed behaviour is caused by a sim-to-real gap, or whether the static error is instead an inherent consequence of the PD-controlled actuators. The resulting commanded and measured joint positions for the front left leg during simulation and unsuspended deployment is shown in Figure 4.4 and Figure 4.5, respectively. In both figures, the static error between the commanded and measured joint position of the knee during an undisturbed state can be observed. During deployment, a change in body velocity can be seen at 0, 4, 6, 9, and 13 seconds which corresponds to the robot being pushed. As a response to these perturbations, the policy and therefore the joint positions change to counteract the force and remain stable. When the pushes are small, the robot recovers to its previous position. When they are large, however, the robot is not able to recover and a change in Spot's configuration can be seen.

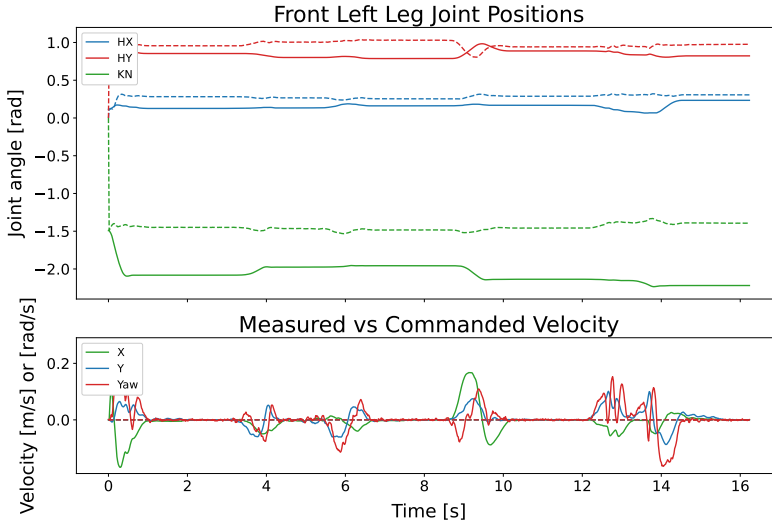


Figure 4.4 Front left leg behaviour during disturbed deployment. Solid line — measured joint position of Spot. Dashed line — desired joint position from policy.

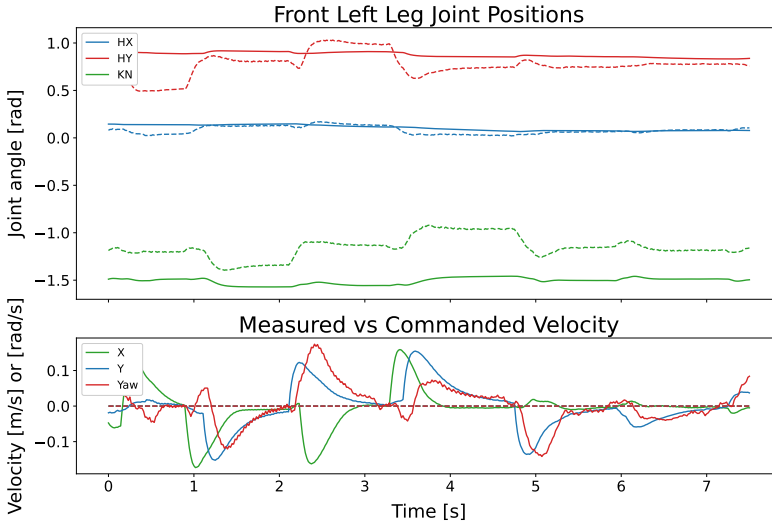


Figure 4.5 Front left leg behaviour during simulated standing. Solid line — measured joint position of Spot. Dashed line — desired joint position from policy.

4.3 Task 2: Locomotion

In this section the different approaches to the locomotion policy are compared. This includes the two-phase architecture, the pre-trained NVIDIA locomotion policy, and the warm-started version. The warm-started locomotion policy will also be deployed, tested, and evaluated on the real hardware.

Training progression: Two-phase

As discussed in Section 3.3, the first locomotion policy developed consisted of two training phases with the goal of improving the speed of convergence and the overall performance. The change in reward structure can clearly be seen in Figure 4.6. When the second phase is entered after 2500 steps, the gait reward declines sharply as a result of the modified reward weight. However, it quickly stabilizes close to 10, the maximum weight of the reward function in the second phase. This suggests that the intention of the first phase, to establish the crucial gait behaviour, was successfully achieved.

The first phase contained extra ground contact sensors that were disabled when entering the second phase in order to prevent terminations that were caused by non-fatal contact with the uneven terrain. These had to be present in early training in order for the agent to not learn a suboptimal strategy, where it made contact with its lower leg to increase stability. The termination causes during training can be seen in Figure 4.7. When switching phase the termination clause for the lower leg ground

contact goes to zero as a result of removing the sensors, where a temporary increase in body contact terminations occurs.

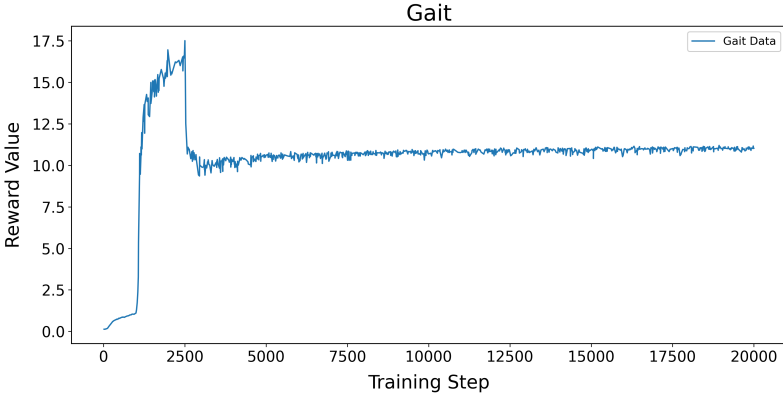


Figure 4.6 Gait reward during training the two-phase locomotion policy.

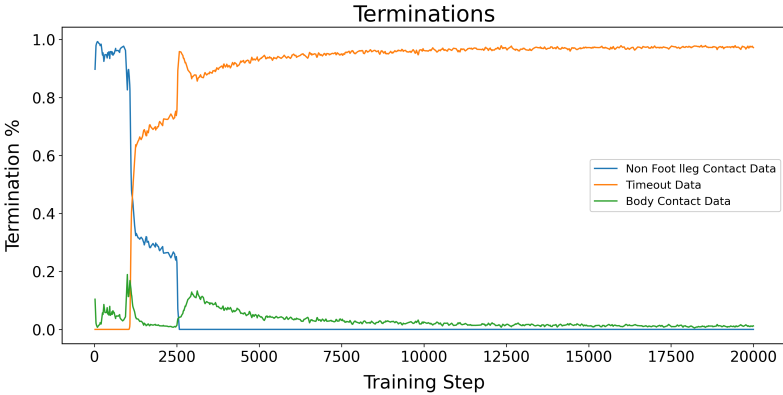


Figure 4.7 Termination causes during training of the two-phase locomotion policy.

Terrain curriculum levels

The terrain curriculum provides additional insight on how the locomotion policies progressed during training. The higher terrain level corresponds to more uneven terrain that is more difficult to navigate. When comparing the training results between the two-phase policy and the warm-started NVIDIA policy, the former reaches a higher curriculum difficulty. This means that the policy is trained on a rougher terrain, which indicates better robustness under rough terrain setups.

4.4 Simulated locomotion test suites

In order to compare the locomotion models in a fair way, a simulation environment was set up that sends the same velocity command input to each model and collects statistics about them that can be seen in Table 4.1 and Table 4.2. The tests consist of two suite of problems, a normal-suite which consists of a series of commands to closely mimic systematic use. Secondly, a stress-suite was used to further test the robustness of the policies. This is done trough increased speeds, harsher changing of directions, and testing the system under different friction coefficients and latencies. A full description of the suites can be seen in Appendix A.1.

Performance differences

Table 4.1 Normal-suite locomotion policy comparison for 15872 simulated tasks.

Policy	Task success [%]	Lin. RMSE [m/s]	Yaw RMSE [rad/s]	Tilt mean [rad]	Foot slip [m/s]	Torque mean [Nm]
NVIDIA baseline	97.3%	0.434	0.119	0.071	0.264	11.43
NVIDIA warm-start	100.0%	0.427	0.126	0.057	0.259	11.10
Two-phase	100.0%	0.399	0.135	0.036	0.297	20.15
Two-phase blended	99.9%	0.370	0.136	0.034	0.266	18.51

Table 4.2 Stress-suite locomotion policy comparison of 64512 simulated tasks.

Policy	Task success [%]	Lin. RMSE [m/s]	Yaw RMSE [rad/s]	Tilt mean [rad]	Foot slip [m/s]	Torque mean [Nm]
NVIDIA baseline	93.4%	0.464	0.245	0.079	0.285	11.60
NVIDIA warm-start	95.0%	0.459	0.282	0.077	0.275	11.16
Two-phase	97.2%	0.462	0.273	0.049	0.340	20.53
Two-phase blended	96.9%	0.424	0.282	0.047	0.295	18.23

Normal suite When examining the data of the normal suite, some conclusions can be drawn. The two-phase policy excels at tracking linear velocities, outperforming the other models in linear RMSE. This is especially apparent when combined with the standing policy, which avoid additional errors during standing tests by preventing the unwanted gaiting behaviour exhibited of the two-phase policy while stationary. However, the improved tracking performance comes at the cost of larger and more aggressive actions. This exemplifies the impact that reward tuning can have on policy performance and highlights the trade-off that occurs when training models.

The warm-started policy is the strongest overall candidate, reaching 100% task success, and having marginal improvements in all categorises expect yaw RMSE over baseline, whilst keeping torques lower.

Stress suite During the stress suite, which was designed to evaluate performance during various disturbances and harsh use-cases, the two-phase achieved higher success rate than both the NVIDIA baseline and the warm-started model while maintaining a lower mean tilt. In contrast, its advantage in linear tracking was reduced to a negligible difference. These results match the expectation, where the two-phase policy utilizes the extra torque it produces to keep the robot steadier.

Was the warm-started policy an improvement? The objective of warm-starting the base policy was to improve stability since the original NVIDIA policy had not been trained with the arm in mind. This was deemed a success since all metrics, outside of yaw RMSE, are improved. The test suites motivate the decision to deploy the warm-started model on hardware.

Qualitatively observed failure cases. In terms of failure cases, diagonal locomotion combined with a commanded angular velocity was found to be the most challenging scenario across all policies. This motion put a large strain on the leg which acts as a pivot point, reducing overall stability. Increasing the magnitude of velocity commands significantly beyond those used during training also increased the imbalance of the system, especially when rapidly oscillating the velocity in opposite directions. When the commanded velocity changes from a high magnitude forward to stopping, it takes a few steps to slow down in order for the standing policy to activate.

4.5 Locomotion deployment

For the hardware deployment of the locomotion policies, both the warm-started policy and the two-phase policy were tested. Additionally, a policy trained before the introduction of the two-phase architecture was tested. This was done using the deployment progression presented under Section 4.1, where the policy was first tested while Spot was suspended in the air. If the suspension test proved successful, the policy could then be tested on the ground, with the harness attached as a source of protection from ground impacts.

Pre two-phase locomotion policy The suspended deployment for the policy trained before the two-phase architecture showed that the policy reacted to the velocity user commands. However, the robot barely moved its joints and the action outputs were observed to be very small. Spot was then descended towards the ground into a position where the robot would support its own weight. This was proved successful, but similarly to the standing policy it held a very low standing position. When the policy was commanded with a velocity the robot was not able to take a step and the behaviour was not fully responsive. Instead of walking the robot rocked towards the direction of the velocity command, but did not manage to lift its leg. Because of this behaviour the policy was not tested further.

Two-phase locomotion policy The suspended deployment of the two-phase policy showed an opposite reaction when compared to the pre two-phase policy. Instead of the non-responsive movement it was very aggressive. The behaviour during training and simulation, where the robot does not stand still when there is no velocity commanded and instead gaits in place, was present during deployment. When observing the action output from the policy and the joint position in the suspended position it was seen that the policy outputted very large actions with desired joint positions outside the range of motion. The result from this test showed a very aggressive robot that moved its joints fast and close to its joint limits. This was an unwanted behaviour and thought to damage the hardware if it would be deployed on the ground. Furthermore, the rapid and aggressive movements also caused the suspension system to swing, something that could further affect the dynamics and resulting actions from the policy. Either way, this policy was not descended towards the ground because of its aggressive movements and the fact that it did not stand still at all. It would not be safe to lower the robot to the ground because it would collide with it which could damage or break the robot. No further testing was conducted.

Initial test of the warm-started policy The warm-started NVIDIA policy was then deployed on Spot while it was suspended in the air. At this point the policy reacted to commanded velocities and showed leg movement reminding of the gaiting behaviour observed in simulation. However, the robot was not always still when the velocity command was zero and still showed signs of walking. These movements, however, are thought to be a result from the change in dynamics of the robot while it is suspended. A situation in which there are no opposing forces acting on the actuators and the suspended robot is able to swing freely. Either way, the observed movement of the robot were neither aggressive nor inactive. The desired joint position from the resulting output of the policy looked reasonable and the movement of the joints followed the positions from the policy. Furthermore, the policy reacted to changes in the robot's body orientation and velocity when it was pushed.

Intermediate test of the warm-started policy Once the previous tests were conducted and showed a promising result, the robot was descended to the ground. While lowering Spot, it was observed how the joint position changed as a result of the added load from the gravitational force when the harness was no longer supporting the robot's weight. At this time the policy was tested again to see how it would react when the user commanded a velocity. Here we could see Spot taking its first steps with the policy that we trained and deployed. It was able to take steps in all different directions, forward, backwards, diagonal, and rotating around itself. It also reacted when pushed, acting towards the force to keep balance.

Final test of warm-started policy Further tests were conducted where the robot was initialized from the ground, stood up using the pre-defined SDK command, and

then transitioned to the warm-started RL policy from the standing position. At this time the harness remained attached solely as a safety measure in the case of balance loss. Under normal operating conditions the suspension system did not support the weight of the robot. However, it could occasionally restrict the full movement of Spot. This should be considered when evaluating the policy, particularly with respect to the robot's robustness against falling over and the movement behaviour. The locomotion policy was tested under different circumstances including walking continuously, walking in short bursts in different directions, and recovery behaviour from externally applied disturbances.

The first test of the policy evaluated Spot's ability to walk continuously in the same direction by sending constant velocity commands for around 30 meters. Forward locomotion was tested using velocity commands in the x-direction, and sideways locomotion was tested using velocity commands in the y-direction. The results showed that the robot was able to walk continuously without falling over and could take multiple steps one after another. The movement behaviour observed during deployment closely resembled the behaviour seen during simulation. Furthermore, the tests demonstrated the policy's ability to handle both the robot's velocity and momentum which built up during the locomotion.

However, Spot was still attached to the safety harness during the experiments which would sometimes restrict its movement. Since the harness was attached to a crane, the crane had to move in unison with Spot to not infer a pulling force on the harness and therefore on the robot. Consequently, the velocity commands could not always be kept constant, to let the crane catch up. Furthermore, full robustness towards falling over could not be evaluated, even though Spot showed no signs of losing balance.

The resulting joint positions of Spot and desired joint positions from the policy output, together with the robots velocity and the commanded velocity during locomotion for Spot's front left leg can be seen in Figure 4.8. When the commanded velocity is constant a clear oscillation pattern in the joint angles can be observed. This corresponds to the repeated movement of taking a step during continuous gaiting. The measured velocity in the corresponding direction fails to reach the commanded velocities. This might be an effect of the suspension harness restricting the Spot's full movement. If this would be tested without the harness the resulting difference between the types of velocities might decrease.

The second test of the policy evaluated its ability to walk in different directions including forward, sideways, diagonal, and rotating around its z-axis. This was done by sending velocity commands along the x, y, and yaw direction as well as combinations of the different directions accordingly. The resulting joint positions of Spot and desired joint positions from the policy output, together with the robot's velocity and the commanded velocity for Spot's front left leg can be seen in Figure 4.9. When comparing the leg joint positions between the experiment for various commanded velocities and continuous commanded velocities, a difference in the pattern is observed. This is a result of the commands changing rapidly and the policy responding

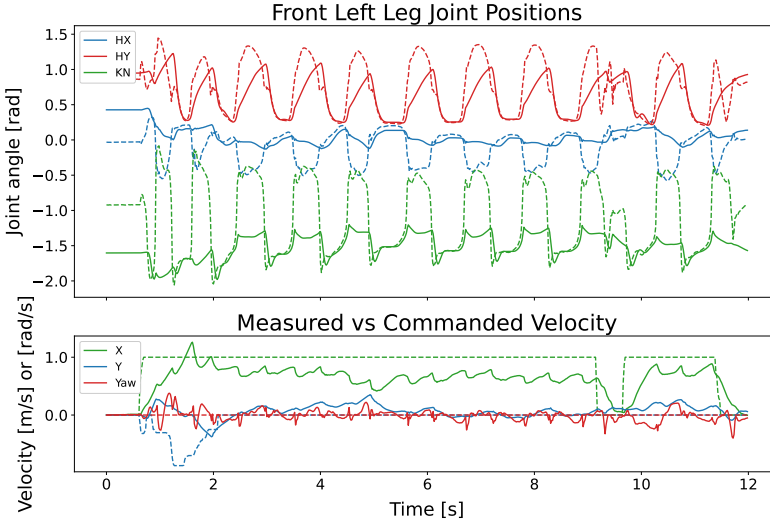


Figure 4.8 Front left leg behaviour during locomotion deployment for continuous forward & sideways walking. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.

dynamically to them.

The last test conducted was to evaluate the policy’s robustness towards external disturbances. This was done by pushing Spot from different angles while it was standing still. When pushed, the output of the policy changed to counteract the applied force by moving the robot’s joints. When necessary, the robot would take a step to maintain balance. Since the harness was still attached the robot could not fall over completely. However, the robot did not show any signs of losing balance and was able to counteract all of the applied disturbances, indicating robustness to external perturbations.

In Figure 4.10, the joint positions of Spot and desired joint positions from the policy output, together with the robots velocity and the commanded velocity for Spot’s front left leg during the pushing experiment are shown. When a force is applied the change in policy behaviour can be observed. The larger movements of the joints between 15 and 20 seconds corresponds to the robot taking a step to counteract the push to find new stable position. The other observed pushed result in a change in the actuator angle, whereafter it returns to its previous position. When compared with the pushing experiments for the standing policy, a more dynamic response to the pushes can be observed. Furthermore, the same gap between the policy’s desired and achieved joint angle is observed during this test, with the tracking error being largest for the knee joints.

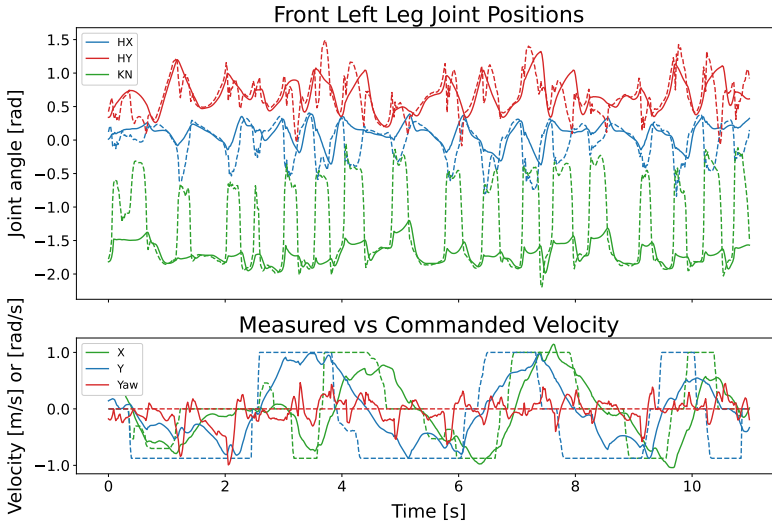


Figure 4.9 Front left leg behaviour during locomotion deployment for various commanded velocities. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.

Overall, the tests conducted for the warm-started locomotion policy demonstrated a behaviour that was robust and similar to that of the simulations. This is a result that proves that deployment of a reinforcement learning policy trained solely in simulation is possible and that the sim-to-real gap is small enough for this to work.

As mentioned, the safety harness was still attached during evaluation and could sometimes hinder the movement of Spot. However, due to the safety of both the hardware and the surrounding, it was chosen to conduct the experiments with this safety precaution.

Interestingly, Spot’s hind left hip joint showed signs of a lower torque output compared to the other legs which was an interesting outcome. It is unclear whether or not this is because of a weaker motor on the real hardware, or if it is an effect of the policy training.

4.6 Summary

In summary, two reinforcement learning tasks were explored. The robust standing policy showed stable recovery behaviour in simulation, but transferred only partially to hardware. Although Spot remained stable and could recover from disturbances, the deployed policy did not reproduce the default standing configuration exactly,

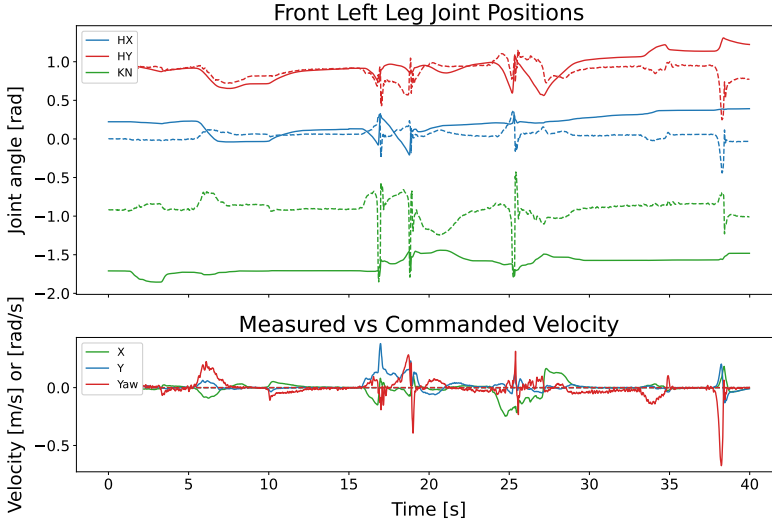


Figure 4.10 Front left leg behaviour from pushing during locomotion deployment. Solid line — measured joint position of Spot / measured body velocity. Dashed line — desired joint position from policy / commanded velocity.

with the largest tracking error observed in the knee joints. For the locomotion task, several policy variants were compared through extensive simulation tests. Based on this comparison, the warm-started NVIDIA policy was selected for deployment. On hardware, this policy was able to produce commanded locomotion in multiple directions and showed robust reactions to external forces, although the experiments remained limited by the use of the safety harness and the observed sim-to-real joint tracking gap.

5

Discussion

This thesis investigated the process of training, validating, and deploying reinforcement learning policies on real Spot hardware using Isaac Sim, Isaac Lab, and an NVIDIA Jetson AGX Orin computer. The discussion is structured around the four research questions and their themes. Firstly, the development and deployment pipeline is discussed, with emphasis on the iterative nature of reinforcement learning design, and the challenges that entail. Secondly, the importance and impact of safety required during deployment. Finally, the observed sim-to-real transfer is discussed, including the limitations caused by model mismatch and joint-tracking errors. Based on the findings, possible future work is proposed towards improving the pipeline and policies.

5.1 Development and deployment pipeline

The development pipeline for reinforcement learning policies proved to be highly iterative. A policy was first trained in Isaac Lab, evaluated in simulation, and then modified based on the observed behaviour. This process had to be repeated several times by adjusting the task formulation, reward weights, termination conditions, and curriculum parameters. The final behaviour of a trained policy was therefore not the result of a single design choice, but of repeated interaction between training, simulation-based evaluation, and finally hardware testing.

A main takeaway from this process is that reward design is both powerful and difficult to predict. Changing a reward term to improve one aspect of the behaviour could worsen another. This was clearly exemplified in the locomotion task, where the policy sometimes continued to move even when commanded to stand still. Attempts to solve this by modifying the reward structure could improve the stand-still behaviour, but at the cost of worse locomotion performance. This demonstrates that the relationship between reward weights and policy behaviour is not linear. A reward change that appears sensible from the task description can lead to unexpected changes in the learned behaviour.

The blending of standing and locomotion policy was one avenue explored to counteract this issue. This reduced the zero-command locomotion behaviour while preserving the locomotion behaviour learned by the walking policy. This was found empirically to produce a stable policy in simulation, but required heavy tuning and was hence deemed a brittle solution, but showed great potential that could be explored further. The result suggests that modular policy composition can be useful when different behavioural modes place conflicting demands on the reward structure, but requires additional work to achieve a satisfactory result.

Another important part of the development pipeline was the choice of robot model. During the thesis, this became an early design decision that affected both training and deployment. The details of how model accuracy influenced the sim-to-real transfer are discussed further in Section 5.3.

Overall, the established pipeline made it possible to train and deploy policies that could perform meaningful behaviours on the real Spot hardware. However, the process also showed that reinforcement learning deployment on hardware is sensitive to task design, model accuracy, and validation procedure. A policy that performs well in simulation is not automatically safe or successful on hardware, and the development loop must therefore include both simulation-based testing and carefully staged deployment.

5.2 Safety during deployment

Safe deployment was a central requirement throughout the thesis. Reinforcement learning policies can produce unexpected joint commands, especially when transferred from simulation to hardware. Since Spot is a heavy and expensive robot with powerful actuators, policy deployment must be performed gradually and with multiple safety layers. The precautions described in *Hardware deployment on Spot* helped validate policy behaviour while minimizing the risk of damaging the robot and the surrounding environment.

Although the harness enabled deployment at an acceptable risk level, it also limited the interpretation of the locomotion results. Since the harness restricted the robot and altered its interaction with the ground, the full movement and robustness of the policy could not be assessed. Nevertheless, the deployed policy successfully achieved locomotion on hardware without damaging the robot. However, it cannot yet be considered fully validated under unrestricted conditions. To fully evaluate robustness, the tests should therefore be conducted without the suspension system.

Other measures taken included decreasing the action scale a_{scale} , producing smaller desired joint positions and therefore lower torque outputs from the PD controller. During suspended deployment, the action scale was initially kept low and increased gradually. At lower scales, unwanted gaiting behaviour would disappear, which was particularly important while descending the robot towards the ground. Decreasing the action scale was especially useful when testing the two-phase loco-

motion policy, as it allowed the general behaviour of the policy to be observed safely despite its aggressive movements. Although changing the action scale is unsuitable for evaluating the actual performance of the policy, it is a useful safety measure during early deployment.

Another safety measure was to keep specific joints at their default position. This helped in two ways. First, the arm joints were locked during deployment to prevent the arm from colliding with the harness and becoming caught in it. This was possible because the policy was trained to keep the arm in the stowed position. Whether this affected the behaviour of the policy remains unknown. The same mechanism also allowed all joints except selected joints to be locked during early testing. This reduced the risk of leg collision while allowing specific aspects of the robot's behaviour to be evaluated in a controlled manner.

Simulation was also a critical tool for evaluating the policy before deployment. With the user-commanded simulation, the movement of the policy could be monitored before testing on hardware. The sim-to-real gap is still a relevant limitation, but if a policy could not walk or maintain balance in simulation, it was considered reasonable to assume that it would not do so in reality either. Policies that performed well in simulation did not always perform equally well on hardware, but policies that performed poorly in simulation were not expected to improve during deployment. Suspended deployment was also tested in simulation by locking the robot base in the air. This produced similar behaviour to the real suspended tests, where the changed dynamics and absence of ground reaction forces caused the robot to gait in the air.

A final safety precaution during RL policy development is to shape the rewards in a way that penalizes unwanted torque, or even terminates episodes when torque outputs become too high. As shown in Section 4.4, the different policies resulted in different mean torque values. This indicates that torque usage depends on the training setup and can therefore be influenced through reward design. However, this remains an open-ended problem, since simply penalizing high joint torques could also worsen desired behaviours.

Overall, ensuring safe deployment is critical when working with RL policies on Spot and has been an important theme throughout this thesis. Multiple measures were taken to improve safety, both during early testing and during final capability tests. However, the locomotion tests conducted with the harness should also be repeated without the harness to fully evaluate the policy. This was not done in this thesis due to safety considerations.

5.3 Sim-to-real transfer

The development of control policies for high-frequency tasks such as locomotion depends strongly on the quality of the simulator. Isaac Sim and Isaac Lab provide powerful tools for training reinforcement learning policies, but the results from this

thesis show that further tuning is required for reliable hardware transfer. If the simulated system differs too much from the real robot, the policy may be pushed outside the distribution it experienced during training, which can lead to unpredictable behaviour.

A sim-to-real gap was observed during deployment, especially in the knee joints. Comparing simulation and deployment showed a static error between the desired joint positions produced by the policy and the measured joint positions of the robot. Some error is expected when using a PD-controlled joint interface, since the desired joint position is converted into actuator torque by the underlying controller. However, the error was larger on hardware than in simulation, which suggests that the simulated actuator behaviour and load-bearing dynamics did not fully match the real robot. This was further supported by increasing the joint gains in simulation. The tracking error decreased, but was not removed completely, indicating that the mismatch cannot be explained only by controller gains. Similar knee-joint discrepancies have also been observed in related work, where the model was tuned using CMA-ES [Miller et al., 2025; Zhu et al., 2025]. This suggests that tuning the simulation model using collected hardware data is an important next step for improving the transfer.

The importance of model accuracy was observed during earlier development with the Spot model with arm available in Isaac Sim. This model required a frame rotation to stand correctly in simulation, but the change introduced a mismatch between the training setup and the deployment setup. During deployment, this led to unpredictable and undesirable behaviour. A similar behaviour was later reproduced in simulation, which indicated that the issue was not only caused by the hardware deployment, but also by the modelling setup itself. This was one of the main reasons for changing to a different Spot model. Furthermore, it showed that small modelling details, such as frame definitions and initial configurations, can have a large effect on sim-to-real transfer. The simulation model should therefore not be treated only as an implementation detail, but as a central part of the transfer problem. The later choice to use a model of Spot that had previously been used for Isaac Lab training and hardware deployment improved the reliability of the pipeline and reduced one possible source of mismatch between simulation and reality.

5.4 Future work

The most important direction for future work is to reduce the sim-to-real gap by using the data collected from the hardware during deployment. This data could be used to tune simulation parameters, actuator gains, or model properties, as described by Miller et al. (2025) and Zhu et al. (2025). This would be especially relevant for the knee joints and other load-bearing parts of the robot, where the largest differences were found.

A second direction is to further investigate the policy architecture and train-

ing setup. The policies in this thesis were trained using relatively standard network structures and reward formulations. Future work could explore different network architectures, recurrent policies, history-based observations, or teacher-student training schemes.

A further direction is to extend the established pipeline from standing and locomotion towards loco-manipulation. This would require adding arm control, object interaction, and task-level observations such as handle pose or contact feedback. The standing and locomotion policies developed in this thesis can therefore be seen as prerequisite components for later door-opening behaviour.

6

Conclusion

This thesis investigated the development and deployment of reinforcement learning policies for the Spot with an attached arm, utilizing Isaac Lab and the Spot RL research kit. A practical simulation-to-hardware pipeline was established and used to train and deploy policies for robust standing and command-following locomotion.

The results show that deployment of reinforcement learning policies trained in simulation is possible on the Spot platform with the arm attached. The robust standing policy transferred partially, maintaining the balance and recovering from small disturbances but showing large joint tracking differences on hardware. The deployed locomotion policy maintained a stronger transfer and was capable of locomotion. Despite this, a sim-to-real gap remains in the load-bearing joints. Safe deployment at this state in development requires careful procedures such as action ramping, joint restrictions, and physical safety constraints.

Overall, the thesis provides a foundation for future work on reinforcement-learning-based loco-manipulation on Spot. The next step is to use hardware data to reduce the sim-to-real gap and extend the established pipeline from standing and locomotion towards arm control, grasping, and full door-opening behaviour.

Bibliography

- Achiam, J. (2018). *Proximal policy optimization*. OpenAI Spinning Up in Deep RL documentation. URL: <https://spinningup.openai.com/en/latest/algorithms/ppo.html> (visited on 2026-02-19).
- Ackerman, E. (28, 2024). *Boston Dynamics unleashes new Spot variant for re-search*. URL: <https://spectrum.ieee.org/boston-dynamics-research-spot> (visited on 2026-04-21).
- Axelrod, B. and W. H. Huang (2015). “Autonomous door opening and traversal”. In: *2015 IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, pp. 1–6. DOI: 10.1109/TePRA.2015.7219686.
- Boston Dynamics (2026a). *About Spot*. URL: https://dev.bostondynamics.com/docs/concepts/about_spot (visited on 2026-03-05).
- Boston Dynamics (2026b). *About the Spot robot*. URL: <https://support.bostondynamics.com/s/article/About-the-Spot-Robot-72005> (visited on 2026-03-05).
- Boston Dynamics (2026c). *Arm and gripper specifications*. URL: https://dev.bostondynamics.com/docs/concepts/arm/arm_specification (visited on 2026-03-05).
- Boston Dynamics (2026d). *Joint control API*. URL: https://dev.bostondynamics.com/docs/concepts/joint_control/readme (visited on 2026-01-26).
- Boston Dynamics (2026e). *Protocol documentation*. URL: https://dev.bostondynamics.com/protos/bosdyn/api/proto_reference# (visited on 2026-05-18).
- Boston Dynamics (2026f). *Spot - the agile mobile robot*. URL: <https://bostondynamics.com/products/spot/> (visited on 2026-03-05).
- Boston Dynamics (2026g). *Spot arm*. URL: <https://bostondynamics.com/products/spot/arm/> (visited on 2026-03-05).

- Boston Dynamics (2026h). *Spot arm specifications*. URL: <https://support.bostondynamics.com/s/article/Spot-Arm-Specifications-151694> (visited on 2026-03-05).
- Boston Dynamics (2026i). *Spot RL researcher kit user guide*. URL: <https://support.bostondynamics.com/s/article/Spot-RL-Researcher-Kit-User-Guide-49965> (visited on 2026-03-05).
- Boston Dynamics (2026j). *Spot SDK*. URL: <https://dev.bostondynamics.com/> (visited on 2026-03-05).
- Boston Dynamics (2026k). *Spot specifications*. URL: <https://support.bostondynamics.com/s/article/Spot-Specifications-49916> (visited on 2026-03-05).
- Boston Dynamics (2026l). *Starting on the right foot with reinforcement learning*. URL: <https://bostondynamics.com/blog/starting-on-the-right-foot-with-reinforcement-learning/> (visited on 2026-03-06).
- Chen, S., B. Zhang, M. W. Mueller, A. Rai, and K. Sreenath (2022). “Learning torque control for quadrupedal locomotion”. DOI: 10.48550/arXiv.2203.05194. arXiv: 2203.05194 [cs.R0]. URL: <https://arxiv.org/abs/2203.05194>.
- Cheng, X., A. Kumar, and D. Pathak (2023). *Legs as manipulator: pushing quadrupedal agility beyond locomotion*. arXiv: 2303.11330 [cs.R0]. URL: <https://arxiv.org/abs/2303.11330>.
- Gustafsson, J. and P. Rosqvist (30, 2023). *Skill-Based Autonomous Door Opening*. Master’s thesis 2023-32. Department of Computer Science, Lund University, Lund, Sweden. URL: <https://lup.lub.lu.se/luur/download?func=downloadFile&recordId=9138654&fileId=9138655> (visited on 2026-01-21).
- Huang, H., A. Loquercio, A. Kumar, N. Thakkar, K. Goldberg, and J. Malik (2024). *Manipulator as a tail: promoting dynamic stability for legged locomotion*. arXiv: 2305.01648 [cs.R0]. URL: <https://arxiv.org/abs/2305.01648>.
- Kumar, L., S. Sortee, T. Bera, and R. Dasgupta (2023). *Enhancing efficiency of quadrupedal locomotion over challenging terrains with extensible feet*. arXiv: 2305.01998 [cs.R0]. URL: <https://arxiv.org/abs/2305.01998>.
- Lejon, M. and F. Sundt (2026). *RLonSpot*. GitHub repository. URL: <https://github.com/DevMarkusLejon/RLonSpot>.
- Martin, J., T. Everitt, and M. Hutter (2016). *Death and suicide in universal artificial intelligence*. arXiv: 1606.00652 [cs.AI]. URL: <https://arxiv.org/abs/1606.00652>.
- Miller, A., F. Yu, M. Brauckmann, and F. Farshidian (2025). *High-performance reinforcement learning on Spot: optimizing simulation parameters with distributional measures*. arXiv: 2504.17857 [cs.LG]. URL: <https://arxiv.org/abs/2504.17857>.

- Mittal, M. et al. (2025). "Isaac Lab: a GPU-accelerated simulation framework for multi-modal robot learning". *arXiv preprint arXiv:2511.04831*. URL: <https://arxiv.org/abs/2511.04831>.
- NVIDIA (2026a). *Creating a manager-based base environment*. URL: https://isaac-sim.github.io/IsaacLab/main/source/tutorials/03_envs/create_manager_base_env.html (visited on 2026-04-23).
- NVIDIA (2026b). *Creating a manager-based RL environment*. URL: https://isaac-sim.github.io/IsaacLab/main/source/tutorials/03_envs/create_manager_rl_env.html (visited on 2026-03-30).
- NVIDIA (2026c). *Curriculum utilities*. URL: <https://isaac-sim.github.io/IsaacLab/main/source/how-to/curriculums.html> (visited on 2026-04-21).
- NVIDIA (2026d). *Data flow between Isaac Sim and Isaac Lab*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/an-introduction-to-robot-learning-and-isaac-lab/02-nvidia-robotics-simulators-and-environments/04-isaac-lab-and-isaac-sim.html> (visited on 2026-03-30).
- NVIDIA (2026e). *How Isaac Lab accelerates reinforcement learning*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-first-robot-with-isaac-lab/02-how-isaac-lab-accelerates-reinforcement-learning.html> (visited on 2026-03-30).
- NVIDIA (2026f). *Isaac Lab*. URL: https://docs.isaacsim.omniverse.nvidia.com/5.0.0/isaac_lab_tutorials/index.html (visited on 2026-03-06).
- NVIDIA (2026g). *Isaac Lab ecosystem*. URL: <https://isaac-sim.github.io/IsaacLab/main/source/setup/ecosystem.html> (visited on 2026-04-21).
- NVIDIA (2026h). *NVIDIA Isaac Lab*. URL: <https://developer.nvidia.com/isaac/lab> (visited on 2026-03-06).
- NVIDIA (2026i). *NVIDIA Isaac Sim*. URL: <https://developer.nvidia.com/isaac/sim> (visited on 2026-03-06).
- NVIDIA (2026j). *NVIDIA Jetson Orin*. URL: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/> (visited on 2026-03-05).
- NVIDIA (2026k). *Reinforcement learning. Getting Started with Isaac Lab*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/an-introduction-to-robot-learning-and-isaac-lab/01-what-is-robot-learning/05-reinforcement-learning.html> (visited on 2026-02-19).

- NVIDIA (2026l). *Reinforcement learning for robots*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-first-robot-with-isaac-lab/01-what-is-reinforcement-learning.html> (visited on 2026-03-11).
- NVIDIA (2026m). *Task design and the Markov decision process*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-first-robot-with-isaac-lab/03-task-design-and-the-markov-decision-process.html> (visited on 2026-03-30).
- NVIDIA (2026n). *Tying it all together*. URL: <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-second-robot-with-isaac-lab/05-tying-it-all-together.html> (visited on 2026-03-30).
- NVIDIA (2026o). *Using the interactive scene*. URL: https://isaac-sim.github.io/IsaacLab/main/source/tutorials/02_scene/create_scene.html (visited on 2026-04-23).
- NVIDIA (2026p). *Welcome to Isaac Lab!* URL: <https://isaac-sim.github.io/IsaacLab/main/index.html> (visited on 2026-03-06).
- NVIDIA (2026q). *What is Isaac Sim?* URL: <https://docs.isaacsim.omniverse.nvidia.com/latest/index.html#> (visited on 2026-03-06).
- NVIDIA (2026r). *Writing an asset configuration*. URL: https://isaac-sim.github.io/IsaacLab/main/source/how-to/write_articulation_cfg.html (visited on 2026-04-23).
- Omotuyi, O., D. Hoeller, and T. Burnham (2024). *Closing the sim-to-real gap: training Spot quadruped locomotion with NVIDIA Isaac Lab*. NVIDIA. URL: <https://developer.nvidia.com/blog/closing-the-sim-to-real-gap-training-spot-quadruped-locomotion-with-nvidia-isaac-lab/> (visited on 2026-03-05).
- OpenAI (15, 2019). *OpenAI five defeats Dota 2 world champions*. URL: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/> (visited on 2026-02-17).
- OpenAI, C. Berner, G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. de Oliveira Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang (2019). “Dota 2 with large scale deep reinforcement learning”. arXiv: 1912.06680. URL: <https://arxiv.org/abs/1912.06680> (visited on 2026-03-15).
- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). “Proximal policy optimization algorithms”. *arXiv preprint arXiv:1707.06347*. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.

- Silver, D., T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis (2018). “A general reinforcement learning algorithm that masters chess, shogi, and go through self-play”. *Science*. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404> (visited on 2026-02-17).
- Stüde, M., K. Nülle, S. Tappe, and T. Ortmaier (2019). “Door opening and traversal with an industrial cartesian impedance controlled mobile robot”. In: *2019 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 966–972. DOI: 10.1109/ICRA.2019.8793866.
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA. URL: <http://incompleteideas.net/sutton/book/RLbook2018.pdf> (visited on 2026-05-12).
- Tan, J., T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke (2018). “Sim-to-real: learning agile locomotion for quadruped robots”. In: *Robotics: Science and Systems XIV (RSS)*. URL: <https://arxiv.org/abs/1804.10332> (visited on 2026-02-17).
- Tobin, J., R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel (2017). “Domain randomization for transferring deep neural networks from simulation to the real world”. *arXiv preprint arXiv:1703.06907*. DOI: 10.48550/arXiv.1703.06907.
- Tuomisto, J., C. Wang, N. Lindgren, and M. Koskinen (3, 2022). *Mobile manipulation using the Boston Dynamics Spot robot*. Final report (ELEC-E8004 Project Work), Project 7. Aalto University. URL: https://wiki.aalto.fi/download/attachments/212893725/Final_report_2022_topic_1-07.pdf?api=v2 (visited on 2026-01-27).
- Zhang, M., Y. Ma, T. Miki, and M. Hutter (2024). *Learning to open and traverse doors with a legged manipulator*. arXiv: 2409.04882 [cs.R0]. URL: <https://arxiv.org/abs/2409.04882>.
- Zhang, T. and H. Mo (2021). “Reinforcement learning for robot research: a comprehensive review and open issues”. *International Journal of Advanced Robotic Systems* **18**:3, p. 17298814211007305. DOI: 10.1177/17298814211007305.
- Zheng, Y., L. Yao, Y. Su, Y. Zhang, Y. Wang, S. Zhao, Y. Zhang, and L.-P. Chau (2024). “A survey of embodied learning for object-centric robotic manipulation”. DOI: 10.48550/arXiv.2408.11537. arXiv: 2408.11537 [cs.R0].
- Zhu, X., Y. Chen, L. Sun, F. Niroui, S. L. Cleac’h, J. Wang, and K. Fang (2025). *Versatile loco-manipulation through flexible interlimb coordination*. arXiv: 2506.07876 [cs.R0]. URL: <https://arxiv.org/abs/2506.07876>.

A

Appendix

A.1 Simulated test-suite task descriptions

In the following is a brief description of the tasks that are contained in the two test suites.

Normal suite

The normal suite is intended to test the locomotion behaviour during normal operations.

1. **Basic commands:** A set of basic commands at low speed, standing still, walking forward, backward, strafing left and right, moving in the diagonal directions, and rotating around its yaw-axis in both directions.
2. **Normal speed sweep:** The robot tracks forward, lateral, and diagonal commands across a moderate speed range.
3. **Normal direction swaps:** The robot alternates between opposite commands, including forward/backward, left/right strafing, diagonal direction changes, and positive/negative yaw.

Stress suite

The stress suite is designed to test robustness and more challenging locomotion commands.

1. **High-speed sweep:** The robot tracks forward, backward, lateral, and diagonal commands over the configured speed range.
2. **Acceleration ramp:** The commanded forward velocity, lateral velocity, and yaw rate are ramped up and then back down.
3. **Stop recovery:** The robot receives a high-motion command followed immediately by a zero command, testing its ability to stop and stabilize.

4. **Push recovery:** External pushes are applied in the positive and negative x and y directions when standing still and walking.
5. **Friction sweep:** The robot is evaluated while the ground-contact friction coefficient is changed across several levels.
6. **Command-latency sweep:** Commanded velocities are delayed by a fixed number of simulation steps before reaching the policy.
7. **Action-latency sweep:** Policy actions are delayed by a fixed number of simulation steps before being applied to the robot.
8. **Rapid direction swaps:** The robot rapidly alternates between opposing forward, lateral, diagonal, and yaw commands.
9. **Endurance tasks:** The robot executes longer continuous forward, lateral, diagonal, and yaw commands.

A.2 Training parameters

For the sake of reproducibility we provide our exact weights for the policy training, exact parameters for everything can be found on the projects GitHub repository [Lejon and Sundt, 2026].

Robust standing

Table A.1 Numerical configuration parameters for the robust standing policy.

Parameter / Reward Term	Standing policy
Rewards	
keep_alive.weight	4.0
keep_alive_ramp.weight	1.0
stand_feet_contact.weight	1.0
base_angular_velocity.weight	5.0
base_angular_velocity.std	1.333
base_linear_velocity.weight	10.0
base_linear_velocity.std	1.0
base_linear_velocity.ramp_rate	0.5
base_linear_velocity.ramp_at_vel	1.0
joint_pos_default_tracking.weight	6.5
joint_pos_default_tracking.std	0.15
action_smoothness.weight	-2.0
air_time_variance.weight	-1.0
base_motion.weight	-2.0
base_orientation.weight	-3.5
stand_height_error.weight	-3.0
stand_height_error.target_height	0.65
foot_slip.weight	-0.5
foot_slip.threshold	1.0
joint_acc.weight	-1.0×10^{-4}
joint_pos.weight	-0.7
joint_pos.stand_still_scale	5.0
joint_pos.velocity_threshold	0.5
joint_torques.weight	-5.0×10^{-4}
joint_vel.weight	-1.0×10^{-2}
joint_vel_arm.weight	-5.0×10^{-3}
joint_deviation_arm.weight	-7.0
joint_acc_arm.weight	-1.0×10^{-5}

Locomotion

Table A.2 Phase 2 curriculum changes in `feet_spot_locomotion_env_cfg.py`. Phase 2 begins after 60,000 environment steps (2500×24).

Parameter / Reward Term	Phase 1	Phase 2
Resampling time range	(3, 9.0)	(4.0, 10.0)
lin_vel_xrange	(−2.5, 2.5)	(−2.5, 3.0)
lin_vel_yrange	(−2.5, 2.5)	(−2.5, 2.5)
ang_vel_zrange	(−1.0, 1.0)	(−1.5, 1.5)
air_time.weight	5.0	5.0
base_angular_velocity.weight	5.0	5.0
base_angular_velocity.std	1.0	$2.0/1.5 \approx 1.33$
base_linear_velocity.weight	8.0	10.0
base_linear_velocity.std	0.5	1.0
base_linear_velocity.ramp_rate	0.5	0.5
base_linear_velocity.ramp_at_vel	1.0	1.0
foot_clearance.weight	0.5	0.5
gait.weight	20.0	12.0
action_smoothness.weight	−1.0	−1.0
air_time_variance.weight	−1.0	−1.0
base_motion.weight	−2.0	−2.0
base_orientation.weight	−3.0	−3.5
stand_height_error.weight	−2.0	−3.0
stand_height_error.target_height	0.65	0.65
foot_slip.weight	−0.5	−0.5
joint_acc.weight	-1.0×10^{-5}	-1.0×10^{-5}
joint_pos.weight	−0.7	−0.7
joint_torques.weight	-5.0×10^{-4}	-5.0×10^{-4}
joint_vel.weight	-1.0×10^{-2}	-1.0×10^{-2}
joint_vel_arm.weight	-5.0×10^{-3}	-5.0×10^{-3}
joint_deviation_arm.weight	−7.0	−7.0

Table A.3 Numerical configuration changes between the NVIDIA base policy and the warm-started locomotion model.

Parameter / Reward Term	NVIDIA baseline	NVIDIA warm-start
Agent		
Learning rate	0.001	0.0003
Entropy coefficient	0.0025	0.0005
Robot asset and actions		
Controlled joints	All joints	12 leg joints
Commands		
Command resampling time	(10.0, 10.0)	(6.0, 10.0)
Standing command fraction	0.1	0.35
lin_vel_x range	(−2, 3)	(−2, 3)
lin_vel_y range	(−1, 1)	(−1, 1)
ang_vel_z range	(−2, 2)	(−2, 2)
Events		
Reset base velocity, x	(−1.5, 1.5)	(−0.5, 0.5)
Reset base velocity, y	(−1.0, 1.0)	(−0.5, 0.5)
Reset base velocity, z	(−0.5, 0.5)	(−0.2, 0.2)
Reset angular velocity, roll/pitch	(−0.7, 0.7)	(−0.25, 0.25)
Reset angular velocity, yaw	(−1.0, 1.0)	(−0.5, 0.5)
Leg joint reset velocity	(−2.5, 2.5)	(−1.0, 1.0)
Push interval	10–15 s	Disabled
Push velocity, x/y	(−0.5, 0.5)	Disabled
Rewards		
Air-time cmd threshold	Not gated	0.12
Gait cmd threshold	Not gated	0.12
joint_pos stand_still_scale	5.0	8.0
joint_pos velocity_threshold	0.5	0.8
stand_base_motion.weight	Not present	−3.0
stand_base_motion cmd threshold	Not present	0.12
stand_joint_vel.weight	Not present	−0.02
stand_joint_vel cmd threshold	Not present	0.12
stand_feet_contact.weight	Not present	0.5
stand_feet_contact cmd threshold	Not present	0.12
Terminations and simulation		
Leg self-collision threshold	Not present	3.0
Decimation	10	10
Episode length	20.0 s	20.0 s